

CLOUDNOTE AI: CLOUD-BASED INTELLIGENT VIDEO NOTE EXTRACTION AND SEMANTIC SEARCH PLATFORM

(CloudNote AI)

A Project Report submitted in partial fulfilment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE AND ENGINEERING

Submitted by

K. Mithilesh Reddy	2320030330
Nitin Tulugu	2320030186
Sargu Sai Chetan	2320030348

Under the guidance of

Dr. G. Neelima

Department of Computer Science and Engineering

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

KONERU LAKSHMAIAH EDUCATION FOUNDATION

(KL University Hyderabad)

(Deemed to be University estd. u/s 3 of UGC Act 1956)

Hyderabad, Telangana, India

A.Y. 2026-2027



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

KONERU LAKSHMAIAH EDUCATION FOUNDATION

(KL University Hyderabad)

(Deemed to be University estd. u/s 3 of UGC Act 1956)

Hyderabad, Telangana, India

CERTIFICATE

This is to certify that the project report entitled **"Cloud-Based Intelligent Video Note Extraction and Semantic Search Platform"** is a bonafide record of the work carried out by **K. Mithilesh Reddy (2320030330)**, **Nitin Tulugu (2320030186)** and **Sargu Sai Chetan (2320030348)**, submitted in partial fulfilment of the requirements for the award of the degree of **Bachelor of Technology in Computer Science and Engineering** of Koneru Lakshmaiah Education Foundation (KL University Hyderabad), Hyderabad, Telangana, India, during the academic year 2026-2027, as part of the course Engineering Capstone Project (23IE4053R/23IE4053A). The work has been carried out under our supervision and has not been submitted elsewhere for the award of any other degree or diploma.

Signature of the Guide
Dr. G. Neelima
Department of CSE
KL University Hyderabad

Signature of the HOD
Dr. P.V. Rao
Head of the Department
Department of CSE
KL University Hyderabad

Signature of the Internal Examiner

Signature of the External Examiner

DECLARATION

We hereby declare that the project report entitled **"Cloud-Based Intelligent Video Note Extraction and Semantic Search Platform"** submitted to Koneru Lakshmaiah Education Foundation (KL University Hyderabad), Hyderabad, Telangana, India, in partial fulfilment of the requirements for the award of the degree of **Bachelor of Technology in Computer Science and Engineering**, is a record of original work carried out by us under the guidance of **Dr. G. Neelima**, Department of Computer Science and Engineering, during the academic year 2026-2027.

We further declare that the work reported in this report has not been submitted, either in part or in full, for the award of any other degree or diploma in this or any other institute or university. All sources of information and the published literature used in this work have been duly acknowledged. The sample lecture recordings shown in this report are illustrative transcripts written for demonstration, and the speech clips used in testing were generated by the team.

Place: Hyderabad, Telangana, India

Date:

Name	**Register No.**	**Signature**
K. Mithilesh Reddy	2320030330	_____
Nitin Tulugu	2320030186	_____
Sargu Sai Chetan	2320030348	_____

ACKNOWLEDGEMENT

We express our sincere gratitude to our project guide, **Dr. G. Neelima**, Department of Computer Science and Engineering, for the guidance, timely reviews and constructive suggestions given throughout this project. The feedback at each review helped us sharpen both the scope of the system and the honest way in which we evaluated it.

We thank Dr. P.V. Rao, Head of the Department of Computer Science and Engineering, for providing the facilities and an encouraging academic environment, and the Dean and the Principal of Koneru Lakshmaiah Education Foundation for the opportunity to undertake this work as part of the Engineering Capstone Project course.

We are grateful to the course coordinators of the Engineering Capstone Project and to all faculty members and technical staff of the Department of Computer Science and Engineering, whose teaching in data structures, information retrieval, software engineering and cloud computing formed the basis of this work.

Finally, we thank our parents and families for their constant support and encouragement, and our friends and classmates who tried the application and gave feedback on its usability.

K. Mithilesh Reddy (2320030330)

Nitin Tulugu (2320030186)

Sargu Sai Chetan (2320030348)

ABSTRACT

Students increasingly learn from series of recorded lectures, yet a recording is rarely self-contained. Speakers say "as we discussed earlier" or "in the next video we will cover this", and a viewer who missed or forgot the other recording reaches a dead end. Existing tools index each video on its own: they can show where a word was spoken, but not which recording explains what another one assumes. This project asks how a cloud platform can turn a personal library of lectures into searchable, verifiable notes while recovering the context that recordings leave out.

The report presents the design, implementation and evaluation of **CloudNote AI**, a web application in which a learner uploads a video or audio file, or imports a transcript. Amazon Transcribe converts speech into timestamped moments; a language model writes a summary, key points, action items and decisions, each linked to its moment; and questions are answered only from the learner's own material, with numbered citations and playable timestamps. Six backward and seven forward phrase patterns detect references between recordings; candidate explanations are suggested from the learner's other videos; and an unresolved reference can be remembered and re-checked after every new upload, producing a single notification when a match appears. A library map shows dependencies, a watch order and a gaps report, and a separately labelled "Know more" function explains topics no recording covers.

The system is deployed on Amazon Web Services (us-east-1) as one CloudFormation stack: CloudFront and a private S3 bucket serve the single-page application; Amazon Cognito authenticates users and an API Gateway JWT authorizer protects every API call; AWS Lambda runs the API and a background worker; DynamoDB stores each user's data in its own partition; EventBridge connects Transcribe to the worker. Because the project's AWS account has an Amazon Bedrock quota of zero, language-model work runs through a fallback chain (Bedrock, then Groq, then Gemini, then an extractive method). All 59 automated tests and all 43 live security and functional checks passed. On the live deployment an 11-second speech clip went from upload to AI notes in a median of 17.3 s, ask-your-notes answered every covered question with citations, and the API function's median server-side time was 14.9 ms. The evaluation is honest about its limits: the sample library is illustrative, and reference recovery has not yet been measured on real lecture series.

Keywords: Lecture Video Retrieval, Speech Recognition, Note Extraction, Retrieval-Augmented Generation, Citation Grounding, Serverless Computing, AWS.

TABLE OF CONTENTS

CERTIFICATE....i
DECLARATION....ii
ACKNOWLEDGEMENT....iii
ABSTRACT....iv
LIST OF FIGURES....ix
LIST OF TABLES....x
LIST OF ABBREVIATIONS....xii
1. INTRODUCTION....1
1.1 Background....1
1.2 Motivation....1
1.3 Problem Statement....2
1.4 Objectives....2
1.5 Scope of the Project....3
1.6 Methodology Overview....3
1.7 Organisation of the Report....4
2. LITERATURE SURVEY....5
2.1 Introduction....5
2.2 Review of Related Work....5
2.3 Comparative Analysis....8
2.4 Summary of Observations....9
2.5 Research Gap Identification....9
3. SYSTEM ANALYSIS AND REQUIREMENTS....11
3.1 Existing System....11
3.2 Proposed System....11
3.3 Feasibility Study....12
3.4 Stakeholders and User Roles....13
3.5 Functional Requirements....13
3.6 Non-Functional Requirements....15
3.7 Hardware and Software Requirements....15
4. SYSTEM DESIGN....17
4.1 Design Goals....17
4.2 System Architecture....17
4.3 Module Design....18
4.4 Data Design....18
4.5 UML Design....21
4.6 Data Flow Diagrams....24

4.7 Security Design..... 24
4.8 User Interface Design..... 25
5. IMPLEMENTATION..... 27
5.1 Technology Stack..... 27
5.2 Project Structure..... 27
5.3 Key Algorithms and Logic..... 28
5.4 Cloud Deployment..... 31
5.5 Module Walkthrough..... 31
6. TESTING..... 38
6.1 Testing Strategy..... 38
6.2 Test Environment..... 38
6.3 Functional and Integration Test Cases..... 38
6.4 Live System Test Cases..... 40
6.5 Security Testing..... 42
6.6 Performance Testing..... 43
6.7 Responsive and Browser Testing..... 44
6.8 Usability Evaluation..... 45
6.9 Summary of Test Results..... 45
7. RESULTS AND DISCUSSION..... 47
7.1 Overview of Results..... 47
7.2 Discussion of Measured Results..... 47
7.3 Operating Cost..... 48
7.4 Achievement of Objectives..... 48
7.5 Comparison with Surveyed Approaches..... 49
7.6 Limitations..... 49
7.7 Answer to the Research Question..... 50
8. CONCLUSION AND FUTURE SCOPE..... 51
8.1 Conclusion..... 51
8.2 Future Scope..... 51
REFERENCES..... 2
APPENDIX A: API ENDPOINT REFERENCE..... 55
APPENDIX B: DEPLOYMENT AND RUN GUIDE..... 56
B.1 Running locally..... 56
B.2 Deploying to AWS..... 56
B.3 Recovery..... 56
APPENDIX C: USER GUIDE..... 57
C.1 Getting started..... 57
C.2 Everyday tasks..... 57

APPENDIX D: PROJECT LINKS.....58

LIST OF FIGURES

Figure 4.1: Deployed system architecture of CloudNote AI on AWS.....17
Figure 4.2: Processing pipeline for one recording.....18
Figure 4.3: Entity-relationship view of the single-table data model.....19
Figure 4.4: Use case diagram of CloudNote AI.....21
Figure 4.5: Domain class diagram.....22
Figure 4.6: Sequence diagram - sign-in and authorised request.....23
Figure 4.7: Sequence diagram - upload, transcription and notes.....23
Figure 4.8: Sequence diagram - grounded question answering with model fallback.....23
Figure 4.9: Sequence diagram - remembered reference and notification.....24
Figure 4.10: Data flow diagram - level 0 (context diagram).....24
Figure 4.11: Data flow diagram - level 1.....24
Figure 4.12: Sign-in page.....26
Figure 4.13: Home dashboard with the sample library.....26
Figure 5.1: Library with the four sample recordings.....32
Figure 5.2: Video page with AI notes linked to moments.....32
Figure 5.3: A real speech clip after Amazon Transcribe and AI notes.....33
Figure 5.4: Ask tab on the video page.....33
Figure 5.5: Ask-your-notes answer with citations.....34
Figure 5.6: Context tab with references and candidate sources.....34
Figure 5.7: Notification after a remembered reference was explained.....35
Figure 5.8: Know more explanation for a reference with no source.....35
Figure 5.9: Library map with dependencies, gaps and follow-ups.....36
Figure 5.10: Notes page.....36
Figure 5.11: Notes vs Video comparison.....37
Figure 5.12: Command bar (Ctrl+K).....37
Figure 6.1: Home at 390 px phone width.....45

LIST OF TABLES

Table 2.1: Comparison of surveyed literature....8
Table 2.2: Research gaps and how this project addresses them....9
Table 3.1: Comparison of existing and proposed systems....12
Table 3.2: Stakeholders and roles....13
Table 3.3: Functional requirements - common and authentication....13
Table 3.4: Functional requirements - library and ingestion....14
Table 3.5: Functional requirements - notes and questions....14
Table 3.6: Functional requirements - context recovery....14
Table 3.7: Non-functional requirements....15
Table 3.8: Hardware requirements....15
Table 3.9: Software requirements....15
Table 4.1: Module decomposition....18
Table 4.2: Sort-key prefixes in the single table....19
Table 4.3: Data dictionary - Video item....20
Table 4.4: Data dictionary - Chunk item....20
Table 4.5: Data dictionary - Notes, Reference and Notification items....20
Table 4.6: Security controls by layer....25
Table 5.1: Technology stack and justification....27
Listing 5.1: Repository layout....27
Table 5.2: Size of the code base (non-blank lines)....27
Listing 5.2: Follow-up patterns - shared/nlp.js (abridged)....28
Listing 5.3: Score combination - backend/src/core/library.js (abridged)....29
Listing 5.4: Chunk centrality - backend/src/core/notes.js....29
Listing 5.5: Citation normalisation and check - backend/src/core/notesask.js....29
Listing 5.6: Provider chain - backend/src/providers/gemini.js and lambda.js (abridged)..30
Listing 5.7: Transcription start - backend/src/api.js (abridged)....30
Listing 5.8: Table and Transcribe event rule - infra/template.yaml (abridged)....31
Table 6.1: Levels of testing....38
Table 6.2: Test environment....38
Table 6.3: Engine test cases (tests/engine.test.mjs)....39
Table 6.4: Backend core test cases (backend/tests/core.test.mjs)....39
Table 6.5: API test cases (backend/tests/api.test.mjs)....40
Table 6.6: Authentication test cases (live)....40
Table 6.7: Isolation, input validation and injection test cases (live)....41
Table 6.8: Storage, transcription and feature test cases (live)....41
Table 6.9: Transport and header test cases (live)....42
Table 6.10: End-to-end API response times from India (ms, reused connection)....43
Table 6.11: Server-side Lambda duration (CloudWatch, 31 October 2026)....43

Table 6.12: Observed pipeline timing on the live deployment....44
Table 6.13: Summary of test results by category....45
Table 7.1: Defects found during the first real deployment....47
Table 7.2: Estimated transcription cost by usage scenario....48
Table 7.3: Objectives, implementation and evidence....48
Table 7.4: Qualitative comparison with surveyed approaches....49
Table A.1: REST API endpoints....55
Listing B.1: Local setup....56
Listing B.2: Deployment commands....56
Table D.1: Project links....58

LIST OF ABBREVIATIONS

AI	Artificial Intelligence
API	Application Programming Interface
ASR	Automatic Speech Recognition
AWS	Amazon Web Services
CDN	Content Delivery Network
CORS	Cross-Origin Resource Sharing
CSP	Content Security Policy
DFD	Data Flow Diagram
DNS	Domain Name System
ER	Entity-Relationship
FR	Functional Requirement
HSTS	HTTP Strict Transport Security
HTTP(S)	Hypertext Transfer Protocol (Secure)
IaC	Infrastructure as Code
IAM	Identity and Access Management (AWS)
JSON	JavaScript Object Notation
JWT	JSON Web Token
LLM	Large Language Model
NFR	Non-Functional Requirement
NLP	Natural Language Processing
OAC	Origin Access Control (CloudFront)
PITR	Point-in-Time Recovery
RAG	Retrieval-Augmented Generation
REST	Representational State Transfer
S3	Simple Storage Service (AWS)
SAM	Serverless Application Model (AWS)
SDK	Software Development Kit
SPA	Single-Page Application
SRT	SubRip Subtitle format
TF-IDF	Term Frequency-Inverse Document Frequency
TLS	Transport Layer Security
UML	Unified Modeling Language
VTT	Web Video Text Tracks format

CHAPTER 1
1. INTRODUCTION

1.1 Background

Recorded lectures have become a primary learning resource. Universities publish weekly lecture videos, online courses are organised as playlists, and teams keep knowledge as meeting recordings. Video is easy to produce but slow to consult: a learner cannot skim it like a book, and a fact said once at minute 37 is effectively lost unless someone wrote it down.

A second, less obvious problem is that recordings depend on each other. A lecturer says "use the database settings from the earlier tutorial" or "we will look at indexes in the next class". When the viewer does not remember, or never watched, the other recording, the reference becomes a dead end. Course platforms index each video separately, so they answer the question "where was this word said?", not "which recording explains what this one assumes?".

Cloud services now make the building blocks of a better experience widely available. Managed speech recognition converts audio into timestamped text; large language models can summarise and answer questions; and serverless platforms let a small team run such a system with no servers to manage and almost no cost when it is idle. CloudNote AI combines these building blocks around the problem of the links between recordings.

1.2 Motivation

The motivation came from three observations made during the literature survey and from our own experience as students. First, research on educational video shows that learners navigate videos heavily and benefit from short, well-signposted segments [1], [2], [3], yet the tools we use treat each video as an isolated file. Second, general-purpose AI summarisers produce fluent text that cannot easily be checked against the recording, and language models are known to state unsupported facts [13]. Third, managed cloud services have matured to the point where a three-person team can deliver a secure, deployed product rather than a prototype of one component.

The project therefore set out to build a complete, working system whose every generated statement points back to a playable moment, and to evaluate it honestly against measurable criteria.

1.3 Problem Statement

The problem addressed by this project is stated as the research question:

"How can a cloud platform turn a library of recorded lectures into searchable, verifiable notes, and recover the context that individual recordings leave out?"

The specific challenges that motivate this question are:

- **Slow review:** finding a single explanation requires scrubbing through long recordings.
- **Unverifiable summaries:** AI-written notes often cannot be traced back to the exact moment they came from.
- **Lost context:** references to earlier or later recordings are not connected, so viewers meet dead ends.
- **Missing explanations:** when the explaining recording is not yet in the library, nothing remembers to look for it later.
- **Fragmented personal notes:** a learner's own notes are kept apart from the recording and are never checked against it.
- **Operational cost:** running speech recognition and AI on servers is expensive and hard for students to operate.

1.4 Objectives

The objectives of the project, as presented at the first project review, are:

1. **Timestamped transcription** - convert uploaded audio and video into timestamped moments using a managed speech service, and accept existing transcripts.
2. **Verifiable notes** - generate a summary, key points, action items and decisions in which every item links to the moment it came from.
3. **Grounded question answering** - answer questions only from the learner's own recordings and notes, with numbered citations and playable timestamps, and decline when the material does not cover the question.

4. **Cross-video context recovery** - detect references to other recordings, suggest candidate explanations, and keep watching for explanations that are not yet in the library.
5. **Secure, scalable cloud deployment** - private per-user data, authenticated access and a serverless architecture deployed as infrastructure as code.
6. **Honest evaluation** - measure correctness, security behaviour and performance on the live deployment and report limitations openly.

1.5 Scope of the Project

- **In scope:** a responsive web application with sign-up and sign-in; upload of eight media formats up to 500 MB and import of SRT, WebVTT, Amazon Transcribe JSON and plain-text transcripts; transcription; AI and extractive notes; search; transcript-grounded answers; ask-your-notes; reference and follow-up detection; remembered references and notifications; Notes vs Video comparison; library map and gaps report; Know more explanations; command bar; deployment on AWS as infrastructure as code; and evaluation of correctness, security, performance and responsiveness.
- **Out of scope:** institution-wide course libraries shared by many students, instructor dashboards, integration with learning management systems, live (streaming) transcription, speaker identification, and a formal learning-outcome study. These are discussed as future work in Chapter 8.
- **Data:** the four sample recordings are illustrative transcripts written by the team to demonstrate the features, and the speech clips used in testing were generated with text-to-speech. No third-party recordings or personal data were collected.

1.6 Methodology Overview

The work was organised into six phases, matching the plan presented at the first review: (1) literature survey and requirement analysis; (2) architecture and data design; (3) the core engine (transcript parsing, notes, search, reference detection) and the web application; (4) cloud deployment and integration with Transcribe, Cognito and the language models; (5) testing, measurement and security checks on the live system; and (6) documentation, this report, the research paper and the final presentation. Within

phases 3 to 5 the team worked iteratively: every feature was built and tested against a local server with in-memory providers, then deployed to AWS with the same code.

1.7 Organisation of the Report

Chapter 2 reviews fifteen published studies on educational video, summarisation, retrieval, grounded generation and serverless computing, and derives the research gaps. **Chapter 3** analyses the existing and proposed systems, assesses feasibility and specifies the requirements. **Chapter 4** presents the system design: architecture, modules, data model, UML diagrams, data flow diagrams, security and user-interface design. **Chapter 5** explains the implementation with excerpts from the source code and the cloud deployment. **Chapter 6** presents the testing strategy and results. **Chapter 7** discusses the results, costs, objectives and limitations. **Chapter 8** concludes and outlines future work. References and appendices (API reference, deployment guide, user guide and project links) follow.

CHAPTER 2
2. LITERATURE SURVEY

2.1 Introduction

The literature survey was carried out in the first phase of the project to understand how researchers have approached learning from video, extracting notes from text, retrieving relevant passages, grounding generated answers, and running such systems in the cloud. Fifteen publications were selected. They fall into five groups: (a) learning from educational video [1]-[3]; (b) speech recognition [4]; (c) summarisation and note extraction [5]-[7]; (d) retrieval and grounded generation [8]-[14]; and (e) serverless cloud computing [15]. Each study is summarised below, followed by a comparison table and the research gaps that this project addresses.

2.2 Review of Related Work

2.2.1 How video production affects student engagement

Guo, Kim and Rubin [1] analysed 6.9 million video-watching sessions from four courses on the edX platform to understand which production decisions keep learners engaged. Shorter videos were markedly more engaging, and the authors recommend splitting lectures into short segments of about six minutes or less, together with informal, personal presentation styles. The study shows that segmenting content matters to learners. It does not address how learners find the segment that explains a concept, or how segments in different videos relate to each other.

2.2.2 Data-driven navigation of educational videos

Kim, Guo, Cai, Li, Gajos and Miller [2] present LectureScape, a lecture video player that uses the aggregated interaction traces of many previous viewers to support navigation, for example by marking frequently replayed moments on the timeline, providing transcript-based search and summarising highlights. Their evaluation shows that such signals help learners find and revisit important moments. The approach depends on crowd interaction data and works within a single video; it does not connect references across recordings in a personal library.

2.2.3 Reducing cognitive load in multimedia learning

Mayer and Moreno [3] describe nine ways to reduce cognitive load in multimedia learning, including segmenting material into learner-paced parts and signalling the organisation of the content. These principles motivate CloudNote AI's timestamped

notes, moments timeline and explicit links between recordings. The paper offers instructional guidance rather than a system, and the learning effect of our features has not been tested in this project.

2.2.4 Robust speech recognition at scale

Radford et al. [4] introduce Whisper, a speech recognition model trained on 680,000 hours of multilingual audio collected with weak supervision. The model transcribes many languages robustly without task-specific fine-tuning. The work shows that general-purpose speech recognition is now accurate enough to make lecture audio searchable. CloudNote AI uses the managed Amazon Transcribe service instead, because it integrates with S3 and EventBridge and needs no model hosting; Whisper remains an alternative for future offline or in-browser transcription.

2.2.5 LexRank: graph-based lexical centrality

Erkan and Radev [5] propose LexRank, which builds a graph of sentences connected by TF-IDF cosine similarity and ranks sentences by their centrality in that graph, selecting the most central ones as an extractive summary. It performed strongly in the DUC summarisation evaluations without training data. CloudNote AI's extractive fallback uses a simpler variant of the same idea, ranking each transcript chunk by its average similarity to all other chunks.

2.2.6 TextRank

Mihalcea and Tarau [6] present TextRank, a graph-based ranking model for text that applies a PageRank-style algorithm to sentence and word graphs for unsupervised keyword and sentence extraction. Together with LexRank it established graph centrality as a robust, unsupervised baseline. Extractive methods guarantee that every sentence exists in the source, which is valuable when notes must be verifiable, but they produce less readable notes than generative models.

2.2.7 Survey of text summarisation techniques

Nenkova and McKeown [7] survey extractive summarisation techniques, including topic representation, frequency-based and graph-based methods, and discuss evaluation. The survey highlights that extractive summaries are faithful to the source but can be incoherent, while abstractive methods are fluent but harder to verify. This trade-off shaped our design: a language model writes the notes when available, but every note must refer to a numbered transcript chunk, and the extractive method is the fallback.

2.2.8 Term weighting for automatic text retrieval

Salton and Buckley [8] compare term-weighting schemes for automatic retrieval and establish the effectiveness of TF-IDF weighting with cosine normalisation. This remains a transparent, fast and dependency-free baseline. CloudNote AI's default search, reference matching and notes comparison all use TF-IDF cosine similarity over stemmed tokens with a small synonym table.

2.2.9 The probabilistic relevance framework: BM25

Robertson and Zaragoza [9] describe the probabilistic relevance framework and the BM25 ranking function, which adds term-frequency saturation and document-length normalisation to term weighting. BM25 is a strong lexical baseline for passage retrieval. Our implementation uses plain TF-IDF for simplicity; replacing it with BM25 is a low-cost improvement noted for future work.

2.2.10 Sentence-BERT

Reimers and Gurevych [10] adapt BERT with siamese and triplet network structures to produce sentence embeddings whose cosine similarity reflects meaning, making semantic search over large collections practical. Embeddings find paraphrases that share no words with the query. CloudNote AI supports semantic search with 512-dimensional Amazon Titan embeddings combined with a 0.2 lexical weight, but this mode is currently disabled because the project's Bedrock quota is zero.

2.2.11 Dense passage retrieval

Karpukhin et al. [11] show that dense passage retrieval with a dual-encoder trained on question-passage pairs outperforms BM25 for open-domain question answering on several benchmarks. The work confirms the value of learned retrieval, but requires training data and an embedding service. It informs our plan to enable embedding-based retrieval once Bedrock is available.

2.2.12 Retrieval-augmented generation

Lewis et al. [12] combine a retriever with a sequence-to-sequence generator so that generated answers are conditioned on retrieved passages (retrieval-augmented generation, RAG). This improves factual accuracy on knowledge-intensive tasks and lets the knowledge source be updated without retraining. CloudNote AI applies the same principle at the scale of one learner's library: answers are generated only from retrieved notes or transcript moments.

2.2.13 Hallucination in natural language generation

Ji et al. [13] survey hallucination, the generation of fluent text that is unsupported by or contradicts the source, across summarisation, dialogue and question answering. They describe causes, metrics and mitigation methods and conclude that hallucination remains an open problem. This motivates our two safeguards: answers must cite numbered passages, and uncited answers are discarded.

2.2.14 Generating text with citations

Gao, Yen, Yu and Chen [14] introduce the ALCE benchmark for evaluating language models that generate answers with citations, measuring fluency, correctness and citation quality. They find that even strong models often fail to fully support their statements with citations. Our system enforces citations structurally, checks that cited numbers exist, and shows the cited moments so the learner can verify them.

2.2.15 Serverless computing

Jonas et al. [15] present the Berkeley view of serverless computing, arguing that functions-as-a-service with managed storage simplify cloud programming and charge only for use, while noting limitations such as cold starts and limited execution time. CloudNote AI adopts this model: Lambda functions for the API and the worker, managed storage and identity, and no servers to manage. We report the cold starts and time limits we met.

2.3 Comparative Analysis

Table 2.1 summarises the fifteen studies in terms of focus, method, main inference and the gap relevant to this project.

Table 2.1: Comparison of surveyed literature

No.	Authors (Year)	Focus	Method / Technology	Inference	Gap
1	Guo et al. (2014) [1]	Engagement with MOOC videos	Analysis of 6.9 million edX sessions	Short segments engage learners	No cross-video navigation
2	Kim et al. (2014) [2]	Navigating lecture videos	LectureScape, crowd interaction traces	Interaction data aids navigation	Single video; needs crowd data
3	Mayer & Moreno (2003) [3]	Cognitive load	Instructional design review	Segment and signal content	Guidance, not a system
4	Radford et al. (2023) [4]	Speech recognition	Weakly supervised model, 680k hours	Robust multilingual ASR	Transcription only
5	Erkan & Radev (2004) [5]	Extractive summary	LexRank graph centrality	Centrality finds key sentences	Less readable notes
6	Mihalcea & Tarau (2004) [6]	Keyword and sentence extraction	TextRank graph ranking	Unsupervised and robust	No question answering

No.	Authors (Year)	Focus	Method / Technology	Inference	Gap
7	Nenkova & McKeown (2012) [7]	Summarisation survey	Survey of techniques	Faithful vs fluent trade-off	No verification in UI
8	Salton & Buckley (1988) [8]	Term weighting	TF-IDF, cosine	Transparent strong baseline	Misses paraphrases
9	Robertson & Zaragoza (2009) [9]	Ranking	BM25	Strong lexical ranking	Lexical only
10	Reimers & Gurevych (2019) [10]	Sentence embeddings	Siamese BERT	Semantic similarity at scale	Needs embedding service
11	Karpukhin et al. (2020) [11]	Passage retrieval	Dense dual encoder	Beats BM25 on QA	Needs training data
12	Lewis et al. (2020) [12]	Grounded generation	RAG	Retrieval improves factuality	Not personal libraries
13	Ji et al. (2023) [13]	Hallucination	Survey	Unsupported text is common	Mitigation open
14	Gao et al. (2023) [14]	Citations	ALCE benchmark	Models miss citations	Enforcement in apps
15	Jonas et al. (2019) [15]	Serverless computing	Position paper	Pay-per-use simplifies ops	Cold starts, limits

2.4 Summary of Observations

Three observations emerge from the survey. First, research on educational video [1]-[3] shows that learners benefit from segmented, well-signalled content and from navigation aids, but existing aids work inside a single video. Second, retrieval and generation research [8]-[14] provides the tools to answer questions from a collection, but also shows that generated text must be grounded and checked [13], [14]. Third, speech recognition [4] and serverless computing [15] have made the whole pipeline affordable for small teams.

2.5 Research Gap Identification

Based on the survey and the problem statement, five research gaps were identified at the first review. Table 2.2 maps each gap to the way it is addressed in this project.

Table 2.2: Research gaps and how this project addresses them

No.	Research gap	Evidence from literature	How CloudNote AI addresses it
1	Navigation aids stop at the edge of one video	[1], [2]	Detects references between recordings, suggests sources and links them on confirmation
2	No memory for explanations that do not exist yet	Not addressed in surveyed work	Remembered references are re-checked after every upload with one notification
3	Generated notes and answers are hard to verify	[7], [13], [14]	Every note and answer cites a numbered moment with a playable timestamp; uncited answers are discarded

No.	Research gap	Evidence from literature	How CloudNote AI addresses it
4	Semantic search needs extra services	[10], [11]	Lexical search by default, Titan embeddings when available, same pipeline
5	Running ASR and AI is costly to operate	[4], [15]	Serverless AWS stack with pay-per-use services and a free-tier AI fallback

In the tools we reviewed we found none that detects references between recordings and remembers unresolved ones for later checking. We do not claim that no such tool exists. The project therefore positions itself as an integrated, deployed and measured system that combines established techniques around this gap.

CHAPTER 3
3. SYSTEM ANALYSIS AND REQUIREMENTS

3.1 Existing System

Today a student reviewing a lecture series typically relies on a combination of manual note-taking, scrubbing through the video, the search box of the hosting platform, and occasionally a general AI chatbot or summariser. Each of these handles one recording at a time.

The main drawbacks of this arrangement are:

- **Slow retrieval:** locating an explanation means scrubbing or rewatching; platform search, where it exists, matches exact words only.
- **Unverifiable AI output:** chatbot summaries are not tied to timestamps, so errors are hard to detect.
- **Disconnected recordings:** references such as "as we saw earlier" are not linked to the recording that explains them.
- **No follow-up tracking:** promises such as "next class we will cover indexes" are never checked.
- **Notes kept apart:** a student's own notes are never compared with what the lecture actually said.
- **Privacy concerns:** uploading recordings to general chatbots offers no per-user isolation guarantees.

3.2 Proposed System

The proposed system, CloudNote AI, is a private, cloud-hosted library for recorded lectures, available in any browser on desktops and phones. It provides:

- Timestamped transcription of uploaded media with Amazon Transcribe, or import of existing transcripts.
- AI-written notes in which every item links to its moment, with an extractive fallback.
- Answers to questions drawn only from the learner's transcripts and notes, with citations and timestamps.

- Detection of references and follow-ups between recordings, suggested sources, and remembered references re-checked after each upload.
- A library map, a watch order and a gaps report; a Notes vs Video comparison; and labelled general explanations for missing topics.
- A serverless AWS deployment with per-user data isolation, authenticated APIs and infrastructure defined as code.

Table 3.1: Comparison of existing and proposed systems

Aspect	Existing practice	Proposed system (CloudNote AI)
Finding an explanation	Scrub or rewatch; exact-word search	Search and AI answer with playable timestamps
Notes	Manual, or untraceable AI summaries	Summary, key points, actions, decisions linked to moments
Verification	None	Numbered citations; uncited answers discarded
Links between recordings	None	Detected references, suggested sources, user confirmation
Missing explanations	Forgotten	Remembered and re-checked after every upload
Own notes	Separate from the lecture	Compared with the video's key points
Privacy	Shared chatbot sessions	Private per-user library behind Cognito
Cost and operation	Servers or paid tools	Pay-per-use serverless stack, one deploy command

3.3 Feasibility Study

3.3.1 Technical feasibility

All components are mature, documented managed services or plain JavaScript: Amazon Transcribe for speech, Cognito for identity, API Gateway and Lambda for compute, DynamoDB for storage, S3 and CloudFront for files and delivery, and EventBridge for events. The same backend code runs locally against in-memory providers and on Lambda against AWS, which made iteration fast. The only technical constraint met was account-level: Bedrock's quota of zero, handled with a provider fallback chain. The project is technically feasible, as the live deployment demonstrates.

3.3.2 Economic feasibility

The software is open source and uses no licensed components. Every AWS service in the design is billed per use, and AWS Cost Explorer reported no measurable charge for the project account in October 2026 (Section 7.3). The dominant cost at scale is Amazon Transcribe, billed per second of audio; importing existing captions as

transcripts avoids it entirely. The system is economically feasible for individuals and small classes.

3.3.3 Operational feasibility

Learners need only a browser and an e-mail address. The interface follows familiar web conventions, a user manual with screenshots is provided, and a command bar gives keyboard access. Deployment, updates and removal are single commands. The system is operationally feasible.

3.3.4 Legal and ethical feasibility

Users upload their own recordings and are responsible for having the right to do so. Every generated statement is labelled by its source. While a third-party language model is used, transcript text is sent to that provider; this is disclosed in the documentation, and users are advised not to upload confidential recordings.

3.4 Stakeholders and User Roles

Table 3.2: Stakeholders and roles

Role	Responsibilities and interests
Learner	Uploads recordings or transcripts, reads notes, asks questions, links references, receives notifications, compares own notes. Sees only their own library.
Maintainer	Deploys, updates and removes the stack, monitors logs and costs, configures model providers and sign-up restrictions.
Instructor (future)	Would publish a shared course library and use gaps reports to see what a series assumes but never explains.

3.5 Functional Requirements

The functional requirements were derived from the objectives and refined during development. They are listed per module in Tables 3.3 to 3.6.

Table 3.3: Functional requirements - common and authentication

ID	Requirement	Priority
FR-C1	The system shall let a user sign up with e-mail and a password of at least 8 characters including a number, and sign in.	High
FR-C2	The system shall reject every API request without a valid token.	High
FR-C3	The system shall keep each user's library private to that user.	High
FR-C4	The system shall optionally restrict sign-up to configured e-mail domains.	Medium
FR-C5	The system shall provide a responsive layout from desktop to phone width and a Ctrl+K command bar.	Medium

Table 3.4: Functional requirements - library and ingestion

ID	Requirement	Priority
FR-L1	The user shall upload mp4, m4a, mp3, wav, flac, ogg, webm or amr files up to 500 MB and choose the spoken language.	High
FR-L2	The user shall import a transcript in SRT, WebVTT, Amazon Transcribe JSON or plain text.	High
FR-L3	The system shall transcribe uploads and store timestamped chunks.	High
FR-L4	The user shall rename, reprocess and delete recordings; deleting removes media and data.	High
FR-L5	The system shall limit each user to 30 recordings.	Medium
FR-L6	The user shall load four sample recordings to explore the system.	Medium

Table 3.5: Functional requirements - notes and questions

ID	Requirement	Priority
FR-N1	The system shall generate a summary, key points, action items and decisions linked to moments, and state whether a model or the extractive method wrote them.	High
FR-N2	The user shall search one recording or the whole library and receive playable moments.	High
FR-N3	The user shall receive an AI answer drawn from transcript moments, with citations and timestamps.	High
FR-N4	The user shall ask questions answered only from their notes, with citations; the system shall decline when the notes do not cover the question.	High
FR-N5	The user shall compare their own notes with the video's key points (covered, possible mismatch, missed).	Medium
FR-N6	The user shall export notes as Markdown and read them in a full-page reader with in-note search.	Medium

Table 3.6: Functional requirements - context recovery

ID	Requirement	Priority
FR-R1	The system shall detect backward references and forward follow-ups in each transcript.	High
FR-R2	The system shall suggest candidate sources from the user's other recordings (later recordings only for follow-ups).	High
FR-R3	The user shall accept a source, pick one manually, undo, or remember a reference.	High
FR-R4	The system shall re-check remembered references after each new recording and notify the user once per match.	High
FR-R5	The system shall show a library map with dependencies, a watch order and a gaps report.	Medium
FR-R6	The user shall request a labelled general explanation of a topic no recording covers.	Medium

3.6 Non-Functional Requirements

Table 3.7: Non-functional requirements

ID	Category	Requirement	Design response
NFR-1	Security - authentication	Only signed-in users reach the API; passwords never stored by the app	Cognito user pool; API Gateway JWT authorizer on every route
NFR-2	Security - isolation	A user can never read or change another user's data	DynamoDB partition key = Cognito user id from the verified token
NFR-3	Security - storage	Media must not be publicly readable	Private, encrypted S3; presigned upload and signed, expiring playback links
NFR-4	Security - transport	All traffic encrypted	HTTPS only, HTTP redirected; HSTS, CSP, X-Frame-Options DENY
NFR-5	Honesty	Generated content must be verifiable and labelled	Citations required; source labels (AI notes, extractive, general explanation)
NFR-6	Performance	Interactive requests complete within a few seconds	Serverless API; per-container index cache; 12 s model budget
NFR-7	Reliability	Model failures must not break the app	Fallback chain Bedrock, Groq, Gemini, extractive
NFR-8	Availability and recovery	Data survives failures and stack deletion	DynamoDB point-in-time recovery; tables and buckets retained on delete
NFR-9	Scalability and cost	Grow with use, cost nothing idle	Lambda, DynamoDB on-demand, S3; per-user limits
NFR-10	Maintainability	Reproducible deployment	One SAM template; deploy and destroy scripts; automated tests
NFR-11	Portability	Runs locally without AWS	Local server with disk storage and dev login

3.7 Hardware and Software Requirements

Table 3.8: Hardware requirements

Component	Development	Deployment / end user
Processor	Any 64-bit CPU (development used Apple silicon)	AWS Lambda on ARM64 (Graviton)
Memory	8 GB RAM or more	API Lambda 1024 MB, worker 1536 MB
Storage	1 GB free for source and dependencies	S3 and DynamoDB, grow on demand
Network	Broadband internet	Any internet connection; HTTPS
Client device	Desktop or laptop	Desktop, laptop, tablet or phone with a modern browser

Table 3.9: Software requirements

Category	Software
Operating system	macOS, Windows or Linux for development; Amazon Linux runtime on Lambda

Category	Software
Runtime	Node.js 22 on Lambda (nodejs22.x); Node.js 20+ locally
Front end	Vanilla JavaScript single-page application, esbuild
Back end	Node.js ES modules, AWS SDK for JavaScript v3
Cloud	CloudFront, S3, Cognito, API Gateway (HTTP API), Lambda, DynamoDB, Transcribe, EventBridge, IAM, CloudWatch, CloudFormation (SAM), Bedrock
External AI	Groq API (openai/gpt-oss-120b), Google Gemini API
Tools	Git and GitHub, AWS CLI, Visual Studio Code, Playwright, Node test runner
Browser	Current Chrome, Edge, Safari or Firefox

CHAPTER 4
4. SYSTEM DESIGN

4.1 Design Goals

The design was guided by five goals derived from Chapter 3: (i) every generated statement traceable to a playable moment; (ii) the learner, not an algorithm, decides what is linked; (iii) authorisation decided by the cloud platform before any application code runs; (iv) serverless, pay-per-use infrastructure defined as code; and (v) one code base that behaves identically locally and in the cloud.

4.2 System Architecture

CloudNote AI is fully serverless and deployed in the AWS us-east-1 (N. Virginia) region. Figure 4.1 shows the deployed architecture.

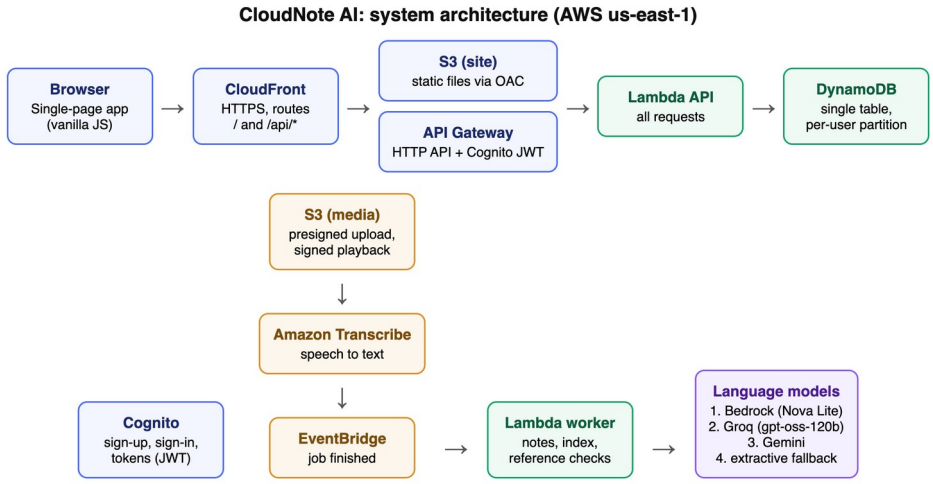


Figure 4.1: Deployed system architecture of CloudNote AI on AWS

Presentation tier. The single-page application is built by esbuild into static files stored in a private S3 bucket and served only through CloudFront using Origin Access Control. A response-headers policy adds HSTS (one year, including subdomains), X-Frame-Options DENY, nosniff, a strict referrer policy and a Content-Security-Policy; plain HTTP is redirected to HTTPS.

Application tier. CloudFront forwards `/api/*` to an API Gateway HTTP API. Its JWT authorizer validates the Cognito token before invoking the API Lambda function (Node.js 22, ARM64, 1024 MB, 29-second limit), so unauthenticated requests never reach application code. A worker Lambda (1536 MB, up to 15 minutes) performs notes, indexing and reference checks asynchronously.

Data tier. One DynamoDB table (on-demand, point-in-time recovery) holds all application data, partitioned by user. Media files and Transcribe output are stored in a private, encrypted S3 bucket; uploads use presigned PUT URLs and playback uses signed, expiring GET URLs.

Events and AI. Amazon Transcribe writes transcripts to S3; an EventBridge rule matches finished jobs named `cn.<user>.<video>.<attempt>` and invokes the worker. Language-model work goes through a fallback chain of Bedrock, Groq and Gemini.

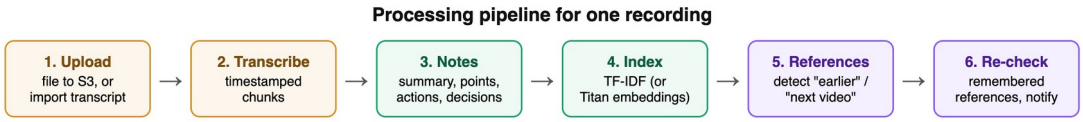


Figure 4.2: Processing pipeline for one recording

4.3 Module Design

Table 4.1: Module decomposition

Module	Responsibility	Main source files
API router	Authorised request handling for all endpoints	backend/src/api.js
Lambda entry points	API, worker and pre-sign-up handlers; provider wiring	backend/src/lambda.js
Pipeline	Transcript to chunks, notes, embeddings, references, pending checks	backend/src/pipeline.js, worker.js
Store	Single-table DynamoDB access per user	backend/src/store.js
Transcript parsing	Transcribe JSON, SRT, WebVTT, plain text	backend/src/core/parse.js
Notes	Model notes with validation; extractive fallback	backend/src/core/notes.js
Library engine	Search, candidates, pending checks, map	backend/src/core/library.js
Grounded answers	Ask-your-notes, transcript answers, citations	backend/src/core/notesask.js
Know more	General explanations	backend/src/core/explain.js
Shared NLP	Tokeniser, TF-IDF, reference patterns, compare	shared/nlp.js
Providers	AWS SDK wrappers; Groq and Gemini clients	backend/src/providers/*.js
Web application	Pages, components, API client, auth	app/js/**
Infrastructure	Stack definition; deploy and destroy	infra/template.yaml, deploy.sh, destroy.sh

4.4 Data Design

4.4.1 Design approach

All application data is kept in one DynamoDB table using a single-table design. The partition key (`pk`) is `U#` followed by the Cognito user id, so every query is naturally

limited to one user and isolation does not depend on application filters. The sort key (`sk`) encodes the item type with a prefix, which lets the API load, for example, all chunks of a recording with one range query. A document store was preferred over a relational database because each user's data is small and self-contained, access patterns are known, and on-demand billing costs nothing when idle.

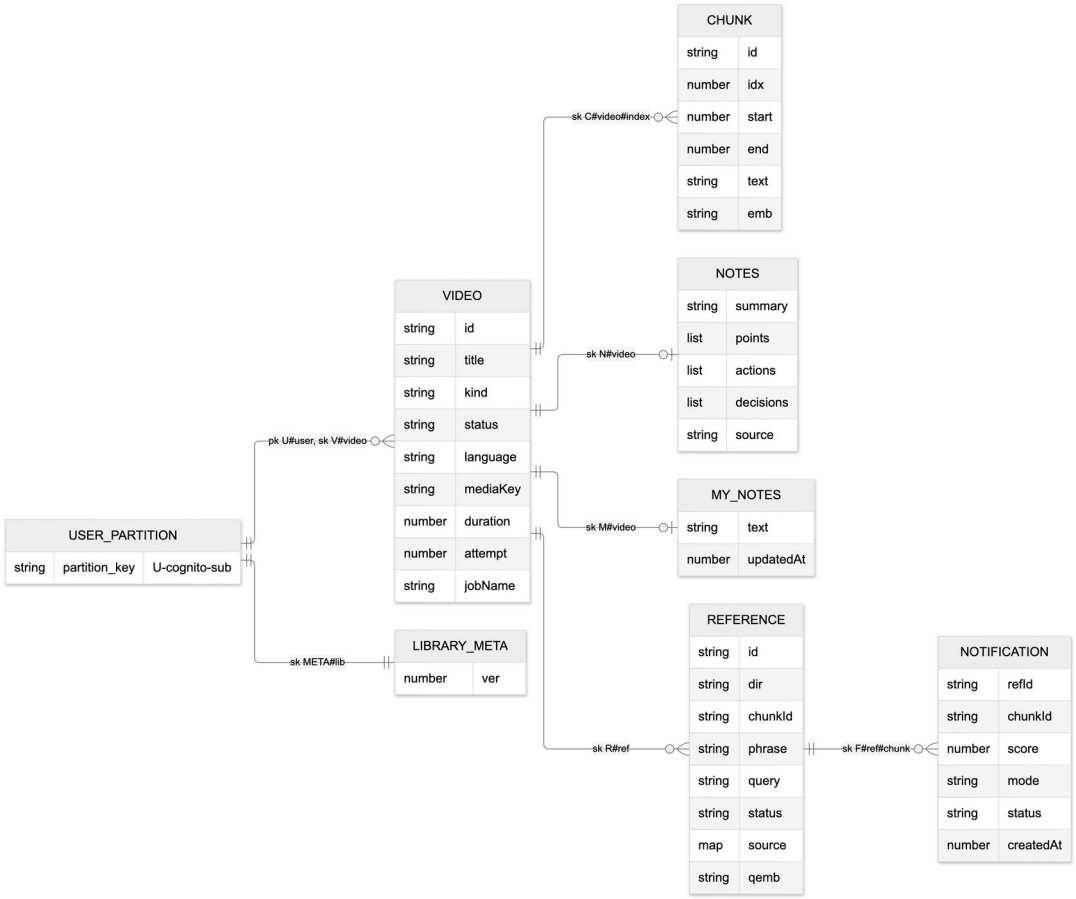


Figure 4.3: Entity-relationship view of the single-table data model

4.4.2 Key design

Table 4.2: Sort-key prefixes in the single table

Item	Sort key	Purpose
Video	V#<video>	Recording metadata and status
Chunk	C#<video>#<index>	Timestamped transcript segment, optional embedding
Notes	N#<video>	Generated notes and their source
My notes	M#<video>	The learner's own notes for Compare
Reference	R#<ref>	Detected reference or follow-up and its state
Notification	F#<ref>#<chunk>	One match for a remembered reference
Library meta	META#lib	Version counter that invalidates cached indexes

4.4.3 Data dictionary

Table 4.3: Data dictionary - Video item

Attribute	Type	Description
id	string	Recording identifier (10 hexadecimal characters)
title	string	Title, up to 120 characters
kind	string	media or transcript
status	string	awaiting_upload, transcribing, processing, ready or failed
language	string	Transcribe language code, default en-US
mediaKey	string	S3 key media/<user>/<video>.<ext>
mediaType	string	MIME type for playback
attempt, jobName	number, string	Transcription attempt and job name cn.<user>.<video>.<attempt>
duration, chunkCount	number	Length in seconds and number of chunks
notesSource	string	model or extractive
createdAt, updatedAt	number	Epoch milliseconds

Table 4.4: Data dictionary - Chunk item

Attribute	Type	Description
id	string	<video>:<index>
idx	number	Position in the transcript
start, end	number	Seconds from the start, null when unknown
text	string	Up to about 60 words
emb	string	Base64 512-dimensional Titan embedding (semantic mode only)

Table 4.5: Data dictionary - Notes, Reference and Notification items

Attribute	Type	Description
summary	string	Notes: short summary
points, actions, decisions	list	Notes: items with text and chunkId
source	string	Notes: model or extractive
dir	string	Reference: back (earlier) or forward (follow-up)
phrase, chunkId	string	Reference: matched phrase and referring chunk
query	string	Reference: referring chunk plus next chunk, meta words removed
status	string	Reference: open, pending or resolved
source	map	Reference: linked chunk and score
refId, chunkId	string	Notification: remembered reference and candidate chunk
score, mode	number, string	Notification: similarity and tfidf or semantic
status	string	Notification: new, accepted or rejected

4.5 UML Design

4.5.1 Use case diagram

Figure 4.4 shows the use cases. The learner is the main actor; the maintainer deploys and operates the stack. Uploading includes transcription and note generation, and reviewing references can be extended by a Know more explanation.

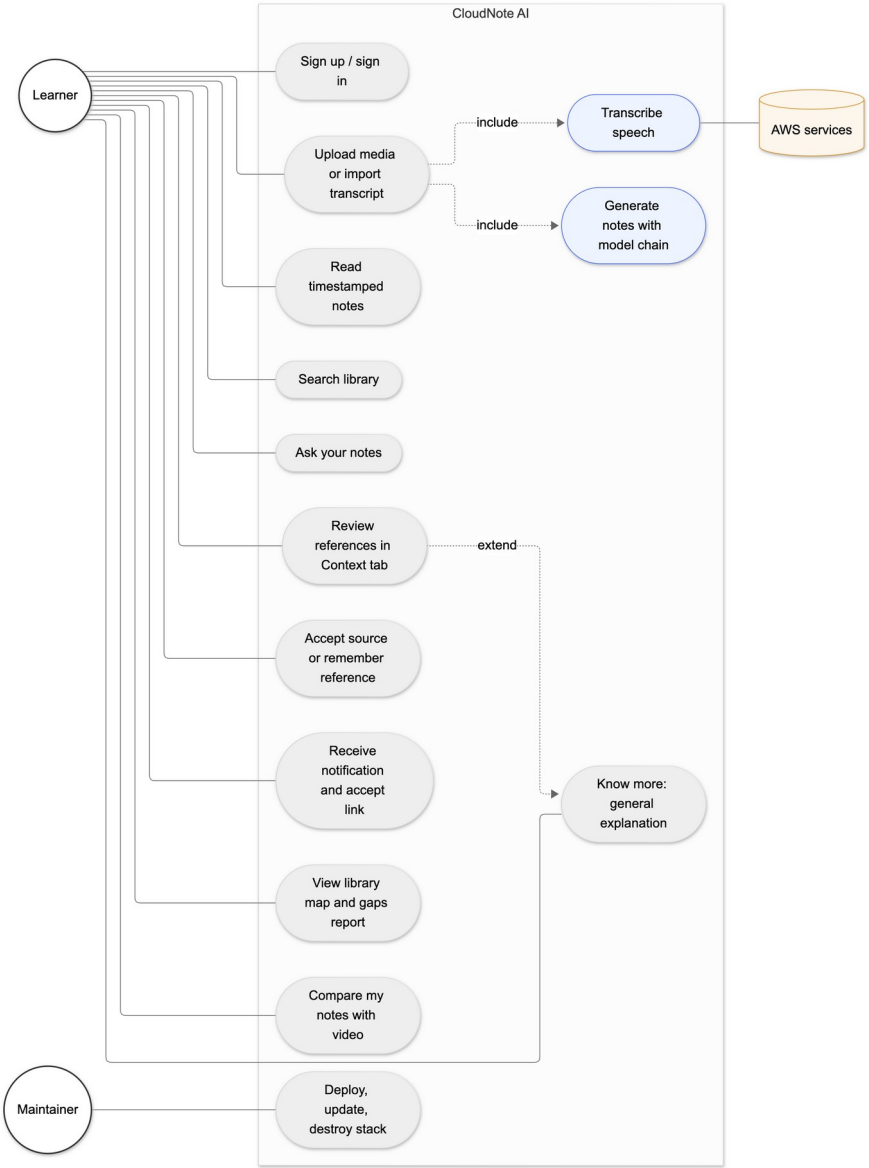


Figure 4.4: Use case diagram of CloudNote AI

4.5.2 Class diagram

Figure 4.5 presents the domain model. A user owns up to 30 recordings; a recording is composed of chunks, notes, the learner's own notes and detected references; a reference may link to a source chunk and owns notifications. Operations correspond to API endpoints of a stateless service.

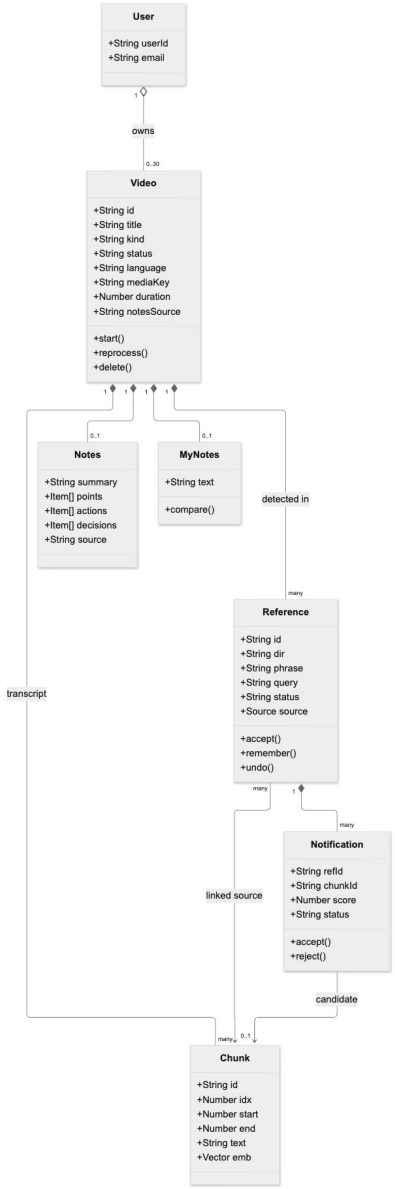


Figure 4.5: Domain class diagram

4.5.3 Sequence diagrams

Figure 4.6 shows sign-up, sign-in and an authorised request: API Gateway rejects a missing or invalid token before any function runs, and the API reads only the caller's partition. Figure 4.7 shows the upload and transcription flow, including the conditional update that prevents a double click from starting two jobs. Figure 4.8 shows a question answered through the model fallback chain, and Figure 4.9 shows a remembered reference producing one notification.

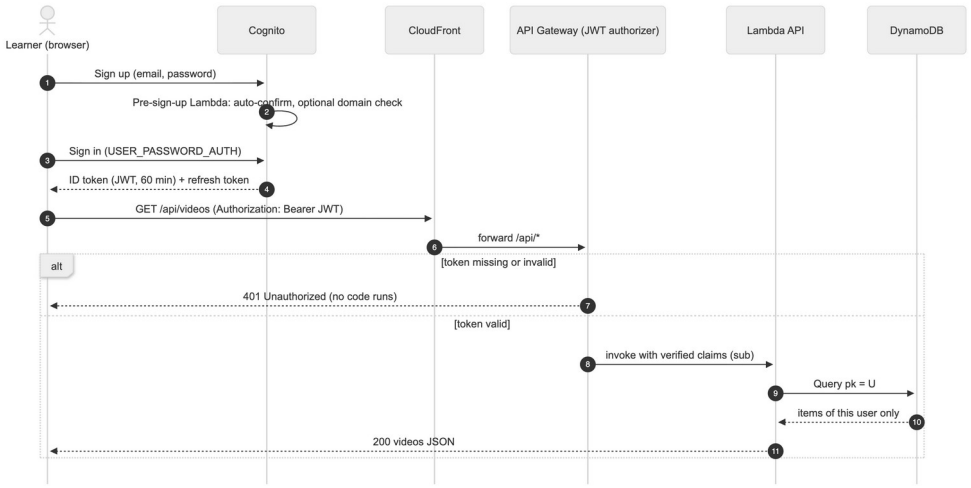


Figure 4.6: Sequence diagram - sign-in and authorised request

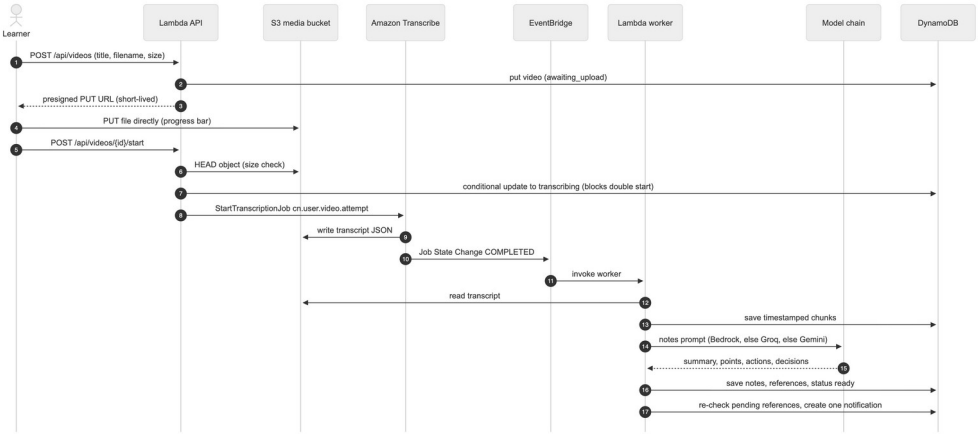


Figure 4.7: Sequence diagram - upload, transcription and notes

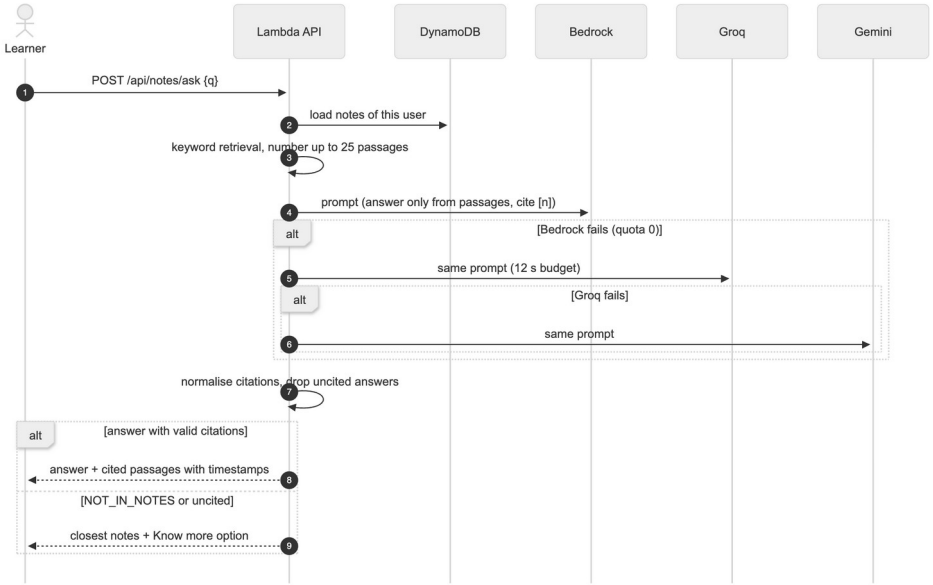


Figure 4.8: Sequence diagram - grounded question answering with model fallback

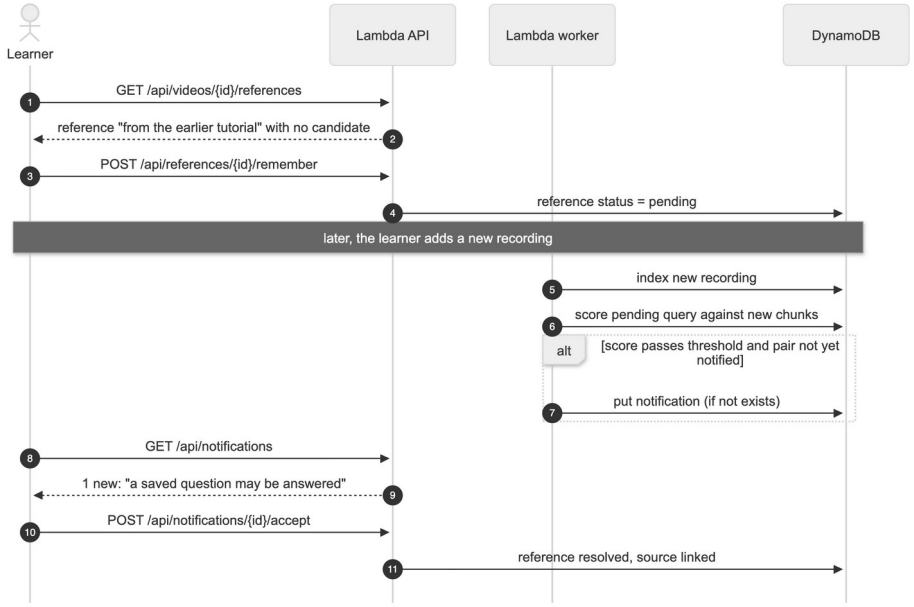


Figure 4.9: Sequence diagram - remembered reference and notification

4.6 Data Flow Diagrams

The level 0 DFD (Figure 4.10) treats CloudNote AI as one process exchanging data with the learner, the maintainer and the external AI providers. The level 1 DFD (Figure 4.11) decomposes it into seven processes and six data stores.

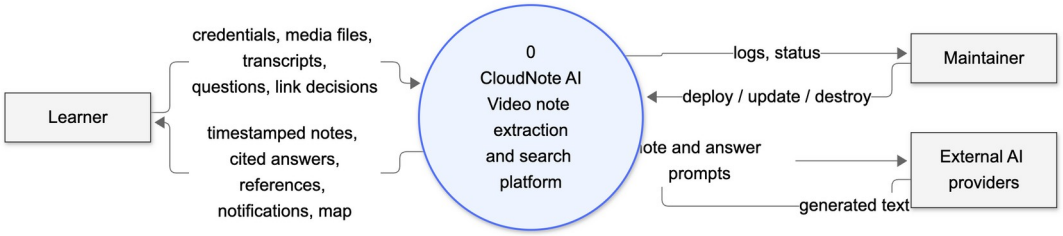


Figure 4.10: Data flow diagram - level 0 (context diagram)

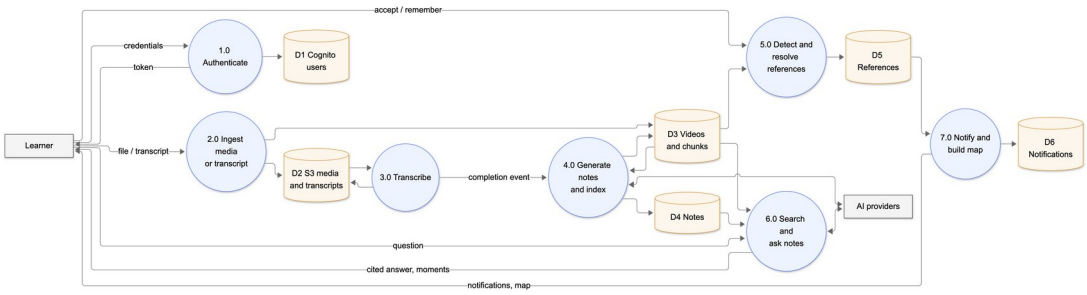


Figure 4.11: Data flow diagram - level 1

4.7 Security Design

Security was designed in independent layers, so that the failure of one control does not expose a learner's data. Table 4.6 summarises them.

Table 4.6: Security controls by layer

Layer	Control	Threat addressed
Identity	Cognito user pool; password policy (8+ characters, number); optional e-mail-domain restriction	Weak or unwanted accounts
API access	API Gateway JWT authorizer on /api/*; 401 before any code runs	Unauthenticated access
Isolation	Partition key from the verified token; no user id accepted from requests	Reading other users' data
Storage	Private S3 buckets with encryption and blocked public access; presigned, expiring URLs; site bucket via CloudFront OAC only	Media or site exposure
Transport and browser	HTTPS only, HSTS, CSP, X-Frame-Options DENY, nosniff, referrer policy	Eavesdropping, clickjacking, script injection
Input	Size and type limits, identifier patterns, transcript length limits	Malformed input, abuse
AI output	Citations required; uncited answers discarded; general explanations labelled	Hallucinated answers
Least privilege	Per-function IAM policies (for example only the worker reads Transcribe results)	Lateral movement
Secrets	Model API keys passed as NoEcho parameters, never in the repository	Key leakage
Recovery	DynamoDB point-in-time recovery; tables and buckets retained on stack deletion	Data loss

4.7.1 Authentication and isolation

Sign-up and sign-in use the Cognito user-pool API. A pre-sign-up Lambda confirms accounts immediately and can enforce an allowed e-mail domain list. The browser sends the ID token as a bearer token; API Gateway validates its signature, issuer, audience and expiry. The API function derives the user id from the verified claims only, so a user can never address another user's partition.

4.7.2 Grounded and labelled AI output

Language models are treated as untrusted authors. Notes must refer to existing chunk numbers or the item is dropped; answers must cite numbered passages or they are discarded; general explanations are visibly labelled as not coming from the learner's videos. This keeps the promise that anything shown as coming from the library can be checked against a timestamp.

4.8 User Interface Design

- **Moment-first:** every note, answer and search result carries a timestamp that plays the recording at that point.

- **Clear provenance:** labels such as AI notes, extractive notes, AI from the video transcript and General AI explanation tell the learner where text came from.
- **User control:** references are suggestions; linking needs an explicit action and can be undone.
- **Progressive disclosure:** the video page groups Notes, Ask, Context and Compare in tabs; the map and reports are one click away.
- **Responsive and keyboard-friendly:** layout adapts down to phone width; Ctrl+K opens the command bar.

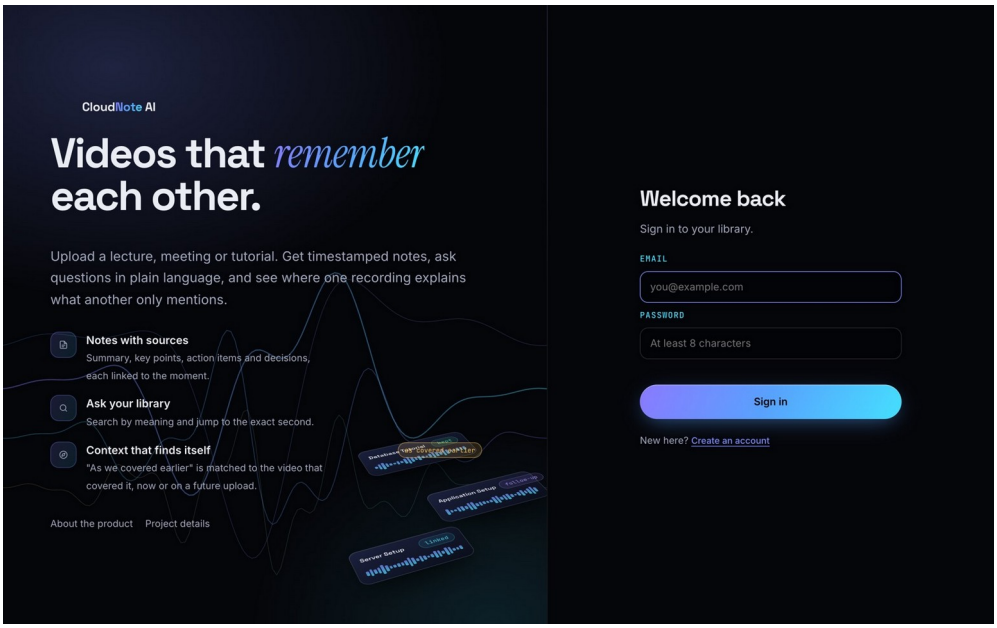


Figure 4.12: Sign-in page

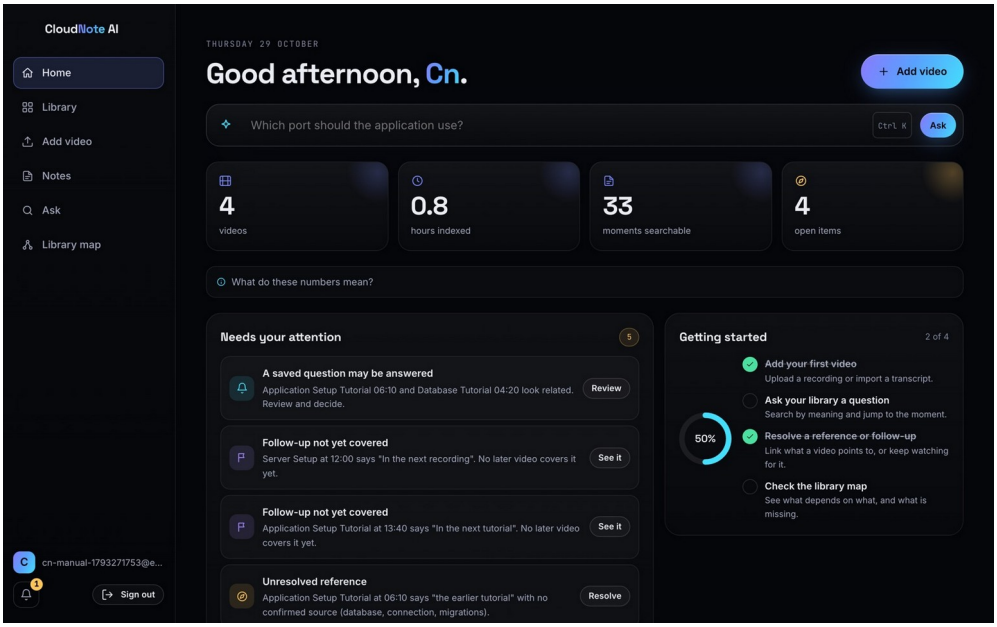


Figure 4.13: Home dashboard with the sample library

CHAPTER 5
5. IMPLEMENTATION

5.1 Technology Stack

Table 5.1: Technology stack and justification

Layer	Technology	Justification
Language	JavaScript (ES modules)	One language for browser, backend and shared engine
UI	Vanilla JavaScript SPA, esbuild	Small bundle, no framework lock-in, fast builds
Backend	Node.js 22 on AWS Lambda (ARM64)	Pay per request, no servers, Graviton price-performance
Identity	Amazon Cognito + API Gateway JWT authorizer	Managed accounts; authorisation before code runs
Database	Amazon DynamoDB (on-demand, single table)	Per-user partitions, idle cost near zero, PITR
Storage and delivery	Amazon S3, CloudFront	Private storage, HTTPS edge delivery, signed links
Speech	Amazon Transcribe + EventBridge	Managed ASR with event-driven completion
Language models	Bedrock, Groq (gpt-oss-120b), Gemini	Resilient fallback chain with free tiers
Infrastructure as code	AWS SAM / CloudFormation	Whole stack reproducible from one template
Testing	Node test runner, Python Playwright	Unit, API and browser tests

5.2 Project Structure

Listing 5.1: Repository layout

```
cloudnote-ai/
|-- app/                web application (index.html, css/, js/ views, api client,
auth)
|-- backend/
|   |-- src/            api.js, lambda.js, worker.js, pipeline.js, store.js,
context.js
|   |   |-- core/       parse.js, notes.js, library.js, notesask.js, explain.js
|   |   `-- providers/  aws.js, gemini.js (Groq + Gemini), local.js, memory.js
|   |-- tests/          core, api, aws and cognito tests
|   `-- local-server.mjs
|-- shared/             nlp.js (tokeniser, TF-IDF, reference patterns), sample-
data.js
|-- infra/              template.yaml, deploy.sh, destroy.sh
|-- src/                project website (about, project pages)
|-- tests/              engine test, Playwright browser scripts
|-- docs/               DEPLOY.md, manual/, paper/
`-- build.mjs           builds site, app and Lambda bundle
```

Table 5.2: Size of the code base (non-blank lines)

Part	Files	Non-blank lines
Web application JavaScript (app/js)	19	1,225
Web application CSS and HTML	2	442
Backend (backend/src)	15	1,276

Part	Files	Non-blank lines
Shared engine (shared)	2	350
Infrastructure (infra)	3	354
Tests (tests, backend/tests)	16	981
Project website (src)	16	2,586

The API exposes 28 endpoints (Appendix A). All of them sit behind the JWT authorizer; the API function additionally rejects requests without a user id, so no route can be added accidentally without authentication.

5.3 Key Algorithms and Logic

5.3.1 Reference and follow-up detection

References are detected with hand-written regular expressions so that every match can be explained by the rule that fired. Six patterns detect backward references and seven detect forward follow-ups; Listing 5.2 shows a subset of the forward patterns.

Listing 5.2: Follow-up patterns - shared/nlp.js (abridged)

```
const FWD_PATTERNS = [
  /\b(?:in|during|for)\s+the\s+(?:next|following|upcoming)\s+(?:video|tutorial|
session|lecture|class|...)s?\b/i,
  /\b(?:next|following)\s+(?:video|tutorial|session|lecture|meeting|class|...)\b/i,
  /\bnext time\b/i,
  /\bin a (?:future|later|follow-?up) (?:video|tutorial|session|lecture|part|...)
\b/i,
  ...
];
export function detectForward(chunks) {
  const found = [];
  chunks.forEach((c, i) => {
    for (const re of FWD_PATTERNS) {
      const m = re.exec(c.text);
      if (m) { found.push({ chunk: c, chunkIndex: i, phrase: m[0], index:
m.index }); break; }
    }
  });
  return found;
}
```

5.3.2 Candidate scoring and remembered references

For each reference the query is the referring chunk plus the following chunk, with words that describe the act of referring (earlier, tutorial, video, ...) removed. Candidates come only from other recordings. In lexical mode a candidate must reach a TF-IDF cosine of 0.30; in semantic mode the score is 0.8 times the embedding similarity plus 0.2 times the lexical score and must reach 0.50. After each new recording is indexed, every pending reference is scored against the new chunks only, and a notification is written with a conditional put so that the same pair is never announced twice.

Listing 5.3: Score combination - backend/src/core/library.js (abridged)

```
export const DEFAULTS = {
  refTfidf: 0.3, refSemantic: 0.5,      // reference matching thresholds
  askTfidf: 0.14, askSemantic: 0.33,    // question thresholds
  semanticLexicalWeight: 0.2,
};
...
if (queryVec && v) {
  score = (1 - this.cfg.semanticLexicalWeight) * dot(queryVec, v) +
this.cfg.semanticLexicalWeight * lex;
  mode = "semantic";
} else { score = lex; mode = "tfidf"; }
hits.push({ chunk: c, score, mode, passes: score >= th[mode] });
```

5.3.3 Extractive notes

When no language model answers, notes are extracted. Each chunk's centrality is its average TF-IDF cosine similarity to all other chunks, a simplified form of LexRank [5]. At most one key point is taken per equal-length segment of the recording (up to eight), the summary is built from the first sentences of the three most central chunks, and action items and decisions are found with keyword patterns.

Listing 5.4: Chunk centrality - backend/src/core/notes.js

```
function centrality(chunks) {
  const idx = buildIndex(chunks.map((c) => ({ id: c.id, videoId: "x", start: 0,
text: c.text })));
  const vecs = idx.docs.map((d) => d.vec);
  return chunks.map((_, i) => {
    let s = 0;
    for (let j = 0; j < vecs.length; j++) if (i !== j) s += cosine(vecs[i],
vecs[j]);
    return s / Math.max(1, vecs.length - 1);
  });
}
```

5.3.4 Grounded answers with citation checking

Both the Ask tab (answers from transcript moments) and ask-your-notes number up to 25 retrieved passages in the prompt and instruct the model to answer only from them, cite them as [n] and reply NOT_IN_NOTES otherwise. Deployed models use several citation styles, so the server normalises them before checking; an answer without a valid citation is discarded rather than shown.

Listing 5.5: Citation normalisation and check - backend/src/core/notesask.js

```
export function parseAnswer(reply, ps) {
  // models vary in citation style: [1, 3], \u30101\u3011, \u30101\u2020L1-L2\u3011,
[1-2] all become [1][3]
  const text = String(reply || "").trim()
    .replace(/[\u3010\[]\s*(\d{1,3})\s*\u2020[^\u3011\]]*[\u3011\]]/g, "[$1]")
    .replace(/[\u3010\[]\s*(\d{1,3}(?:\s*[,;-]\s*\d{1,3})+)\s*[\u3011\]]/g, (_, l)
=> l.split(/\s*[,;-]\s*/).map((n) => `[${n}]`).join(""))
    .replace(/\u3010\s*(\d{1,3})\s*\u3011/g, "[$1]");
  if (!text || /^NOT_IN_NOTES\b/i.test(text)) return null;
  const cites = [...new Set([...text.matchAll(/\[(\d{1,3})\]/g)].map((m) =>
Number(m[1]))
    .filter((n) => n >= 1 && n <= ps.length))];
  if (!cites.length) return null; // an uncited answer cannot be verified, so it is
```

```
not shown
  return { text: text.slice(0, 900), cites };
}
```

5.3.5 Language-model fallback chain

On first deployment the account's Bedrock quota was zero tokens per day for every model. The backend therefore wraps the providers in a chain that returns the first successful answer. Each provider has a time budget (12 s in the API so a request finishes inside API Gateway's 29-second limit, and 60 or 240 s in the worker) and retries with back-off on rate-limit and server errors. Bedrock resumes automatically when the quota is raised.

Listing 5.6: Provider chain - backend/src/providers/gemini.js and lambda.js (abridged)

```
export function chainLlm(models) {
  const list = models.filter(Boolean);
  return { async complete(req) {
    let last;
    for (const m of list) {
      try { return await m.complete(req); }
      catch (e) { last = e; console.error("model failed, trying the next one:",
e.message); }
    }
    throw last;
  } };
}
// lambda.js
llm: chainLlm([ awsLlm({ modelId: NOTES_MODEL_ID, region: BEDROCK_REGION }),
                openAiLlm({ apiKey: GROQ_API_KEY, model: GROQ_MODEL }),
                geminiLlm({ apiKey: GEMINI_API_KEY, model: GEMINI_MODEL }) ]),
```

5.3.6 Starting a transcription safely

The start endpoint checks that the file really reached S3 and is within the size limit, then reserves the recording with a conditional update before calling Transcribe, so that a double click or a retried request cannot start two jobs. Each attempt gets its own job name because job names are unique per account.

Listing 5.7: Transcription start - backend/src/api.js (abridged)

```
const head = await ctx.storage.head(video.mediaKey);
if (!head) throw bad("The upload did not finish. Try uploading again.");
if (head.size > ctx.config.maxMediaBytes) { await
ctx.storage.delete(video.mediaKey); throw bad("That file is too large."); }
const attempt = (video.attempt || 0) + 1;
const jobName = `cn.${userId}.${video.id}.${attempt}`;
// reserve first so a double click cannot start two jobs
if (!(await S.patchVideo(video.id, { status: "transcribing", jobName, attempt, size:
head.size },
    { ifStatus: ["awaiting_upload", "failed"] }))) throw bad("Transcription already
started.");
await ctx.transcribe.start({ jobName, mediaKey: video.mediaKey,
    outputKey: `transcripts/${userId}/${video.id}.json`, language:
video.language });
```

5.4 Cloud Deployment

All resources are declared in one SAM template (infra/template.yaml) and deployed as the CloudFormation stack `cloudnote-ai`. The deploy script builds the site and Lambda bundle, packages the functions, deploys the stack, writes the site configuration (user pool and client id), uploads the site and invalidates the CloudFront cache. Listing 5.8 shows the table and the event rule.

Listing 5.8: Table and Transcribe event rule - infra/template.yaml (abridged)

```
Table:
  Type: AWS::DynamoDB::Table
  DeletionPolicy: Retain
  Properties:
    PointInTimeRecoverySpecification: { PointInTimeRecoveryEnabled: true }
    BillingMode: PAY_PER_REQUEST
    KeySchema: [ { AttributeName: pk, KeyType: HASH }, { AttributeName: sk, KeyType:
RANGE } ]
WorkerFunction:
  Events:
    TranscribeDone:
      Type: EventBridgeRule
      Properties:
        Pattern:
          source: [aws.transcribe]
          detail-type: [Transcribe Job State Change]
          detail: { TranscriptionJobStatus: [COMPLETED, FAILED],
                    TranscriptionJobName: [ { prefix: "cn." } ] }
```

The deployed system is available at https://d349qer3boe6fg.cloudfront.net in region us-east-1, and the source code at https://github.com/kmithileshhhhh05/cloudnote-ai.

5.5 Module Walkthrough

This section presents the implemented modules through screenshots of the live deployment with the sample library.

5.5.1 Library and video page



Figure 5.1: Library with the four sample recordings

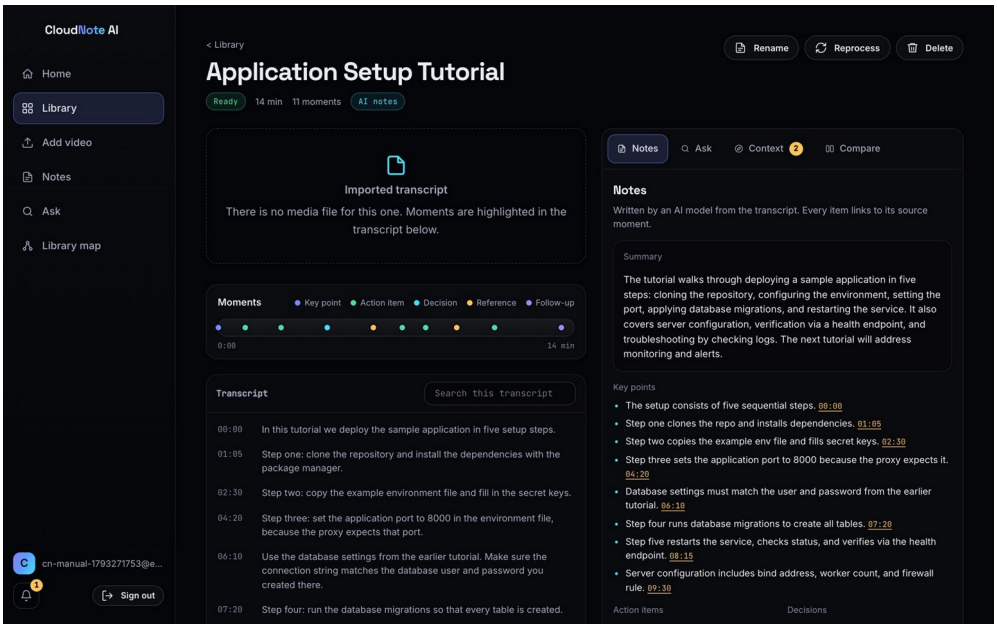


Figure 5.2: Video page with AI notes linked to moments

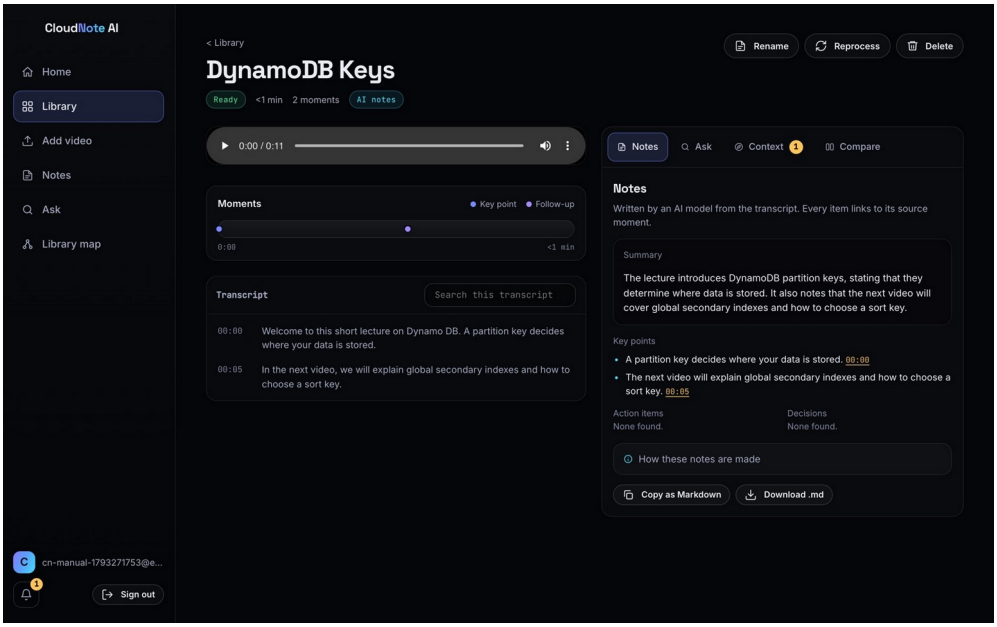


Figure 5.3: A real speech clip after Amazon Transcribe and AI notes

5.5.2 Questions and answers

The Ask tab answers a question from the recording's transcript, citing the moments it used and listing them with timestamps; for example, asking "what is DFS" on a graph-traversal lecture returns a short explanation of depth-first search followed by the moments at 9:30, 12:10 and 15:36 that it came from. Ask-your-notes answers from the learner's notes in the same way.

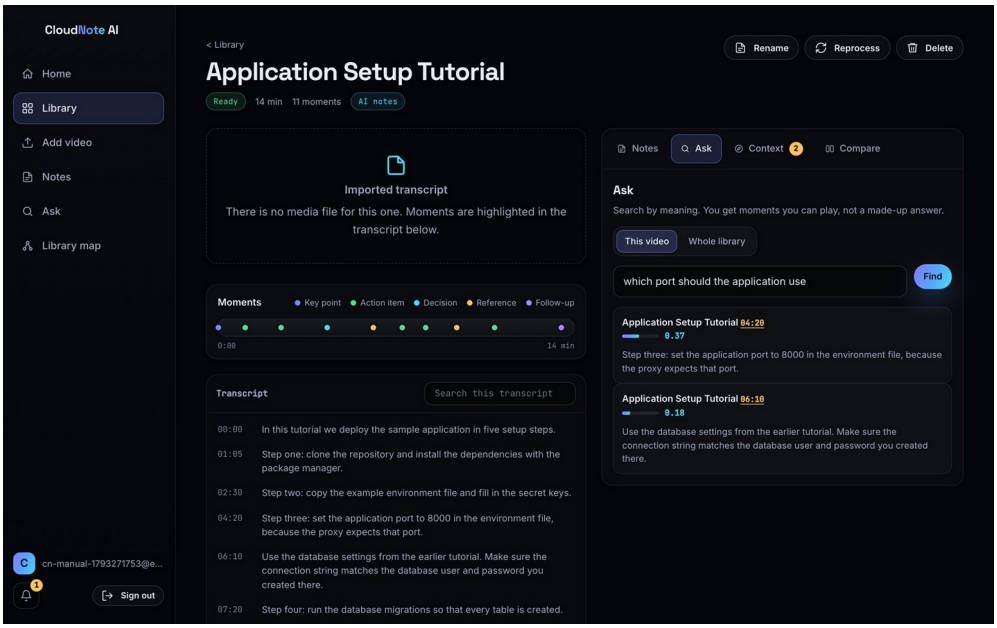


Figure 5.4: Ask tab on the video page

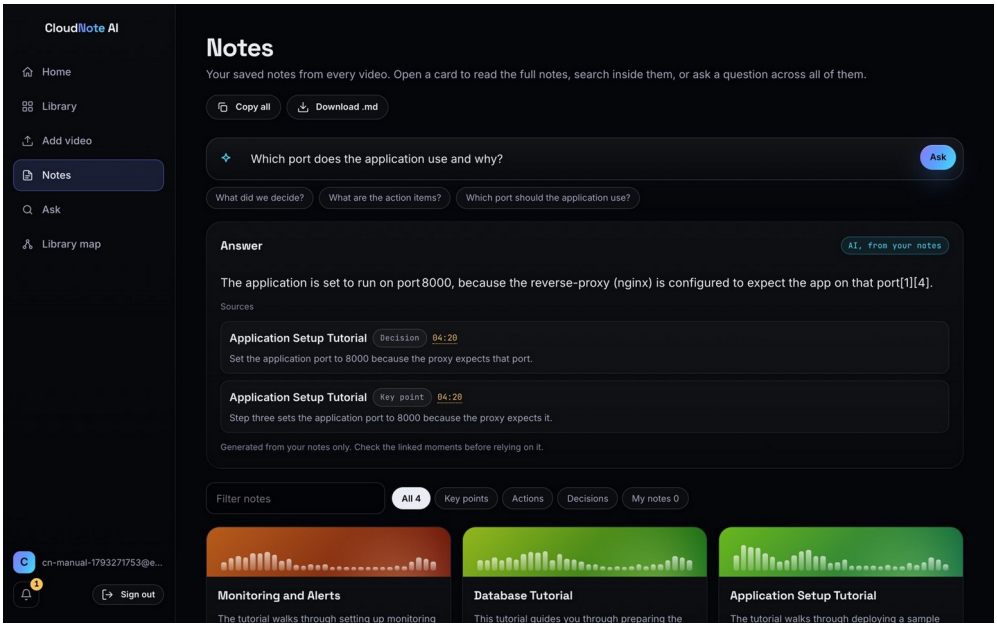


Figure 5.5: Ask-your-notes answer with citations

5.5.3 Context recovery

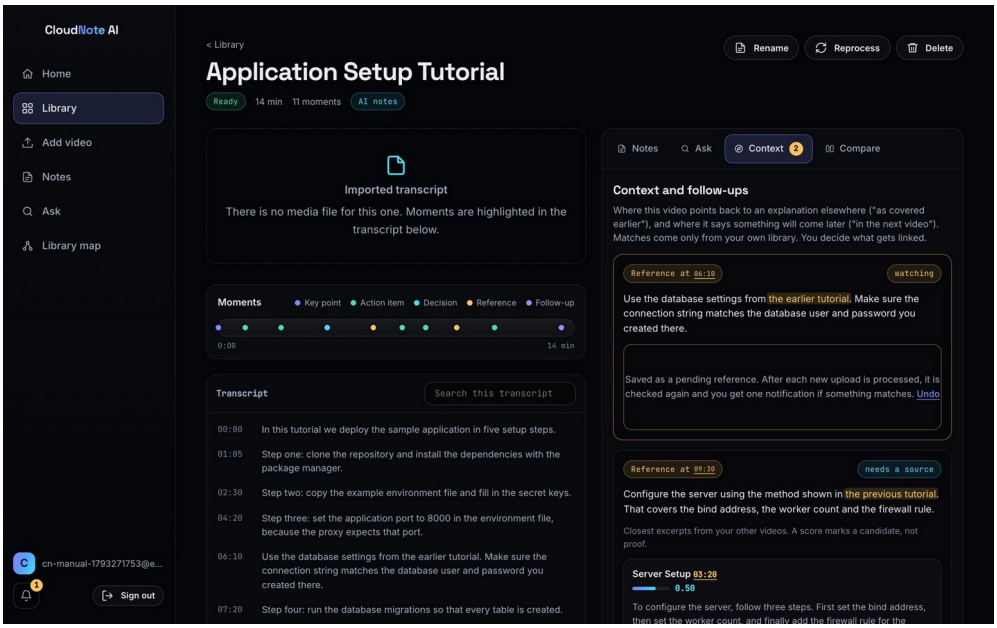


Figure 5.6: Context tab with references and candidate sources

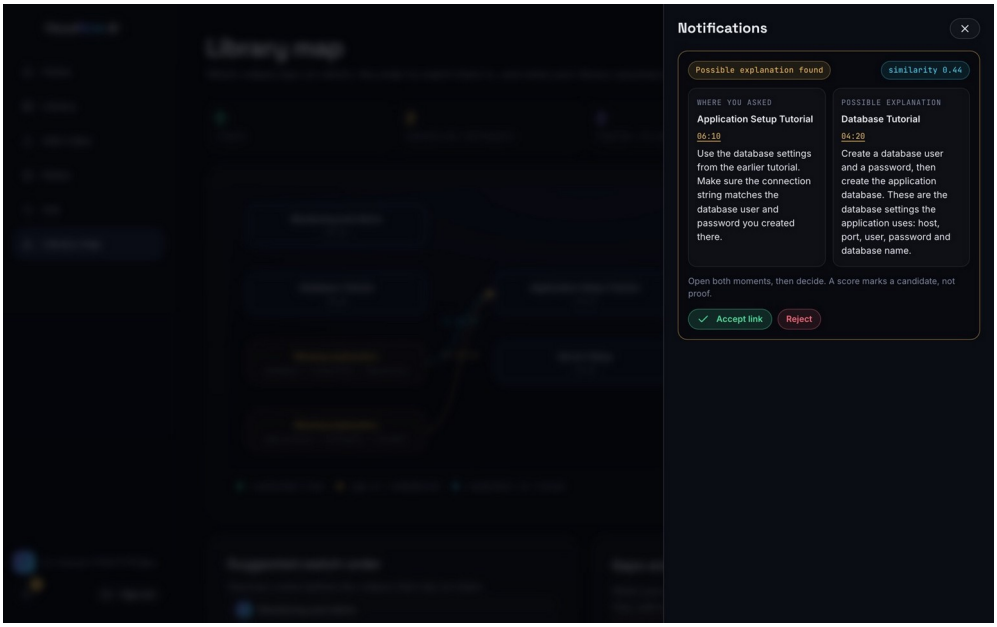


Figure 5.7: Notification after a remembered reference was explained

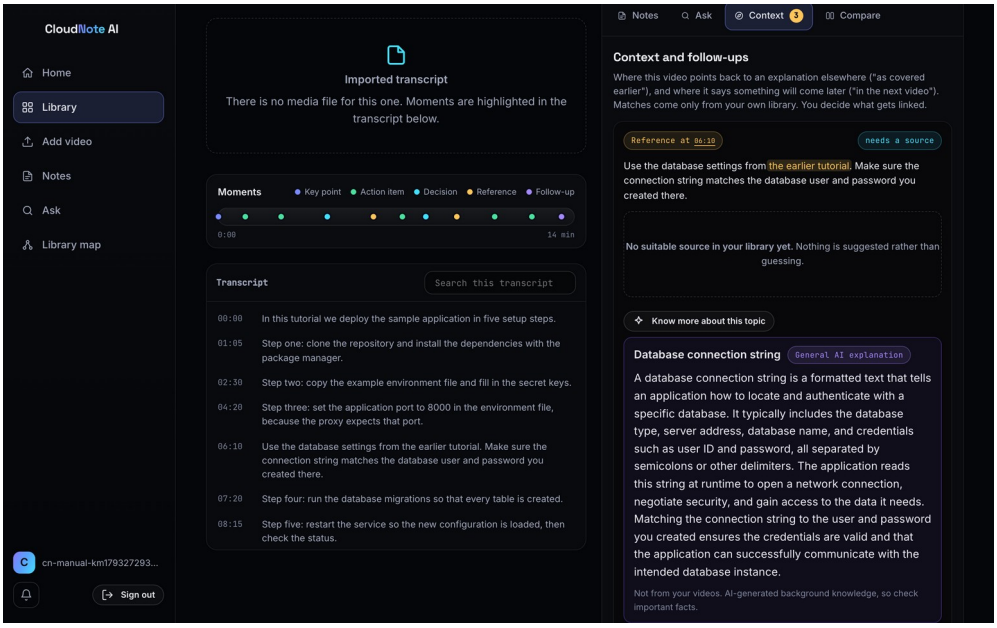


Figure 5.8: Know more explanation for a reference with no source

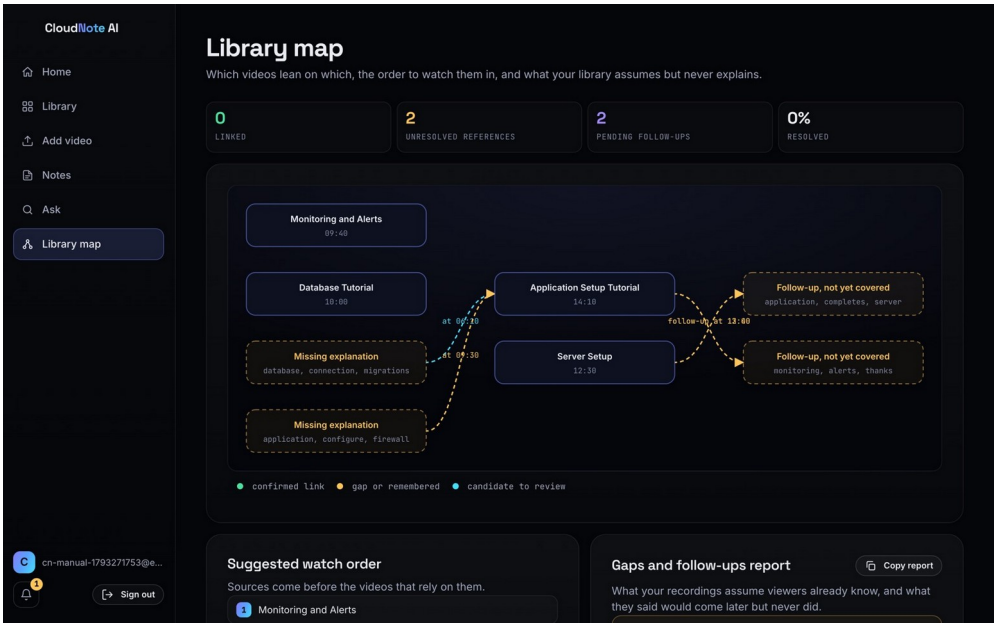


Figure 5.9: Library map with dependencies, gaps and follow-ups

5.5.4 Notes, comparison and command bar

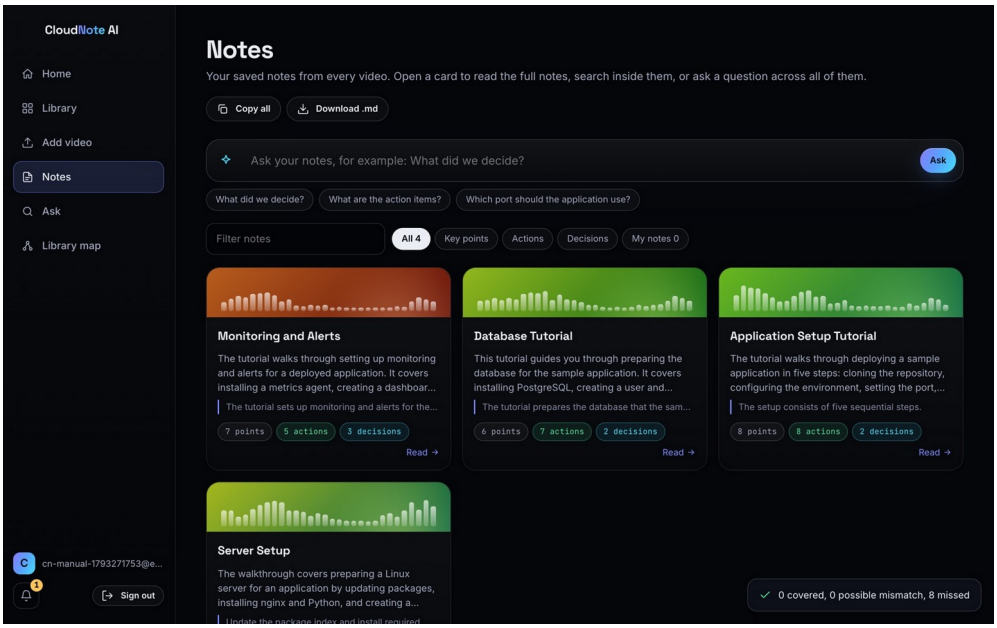


Figure 5.10: Notes page

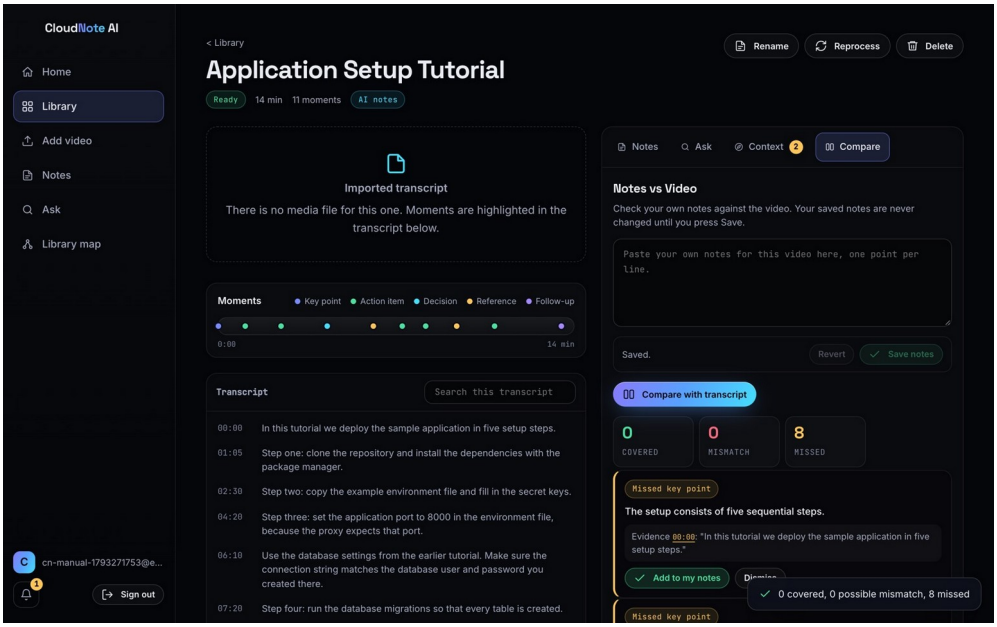


Figure 5.11: Notes vs Video comparison

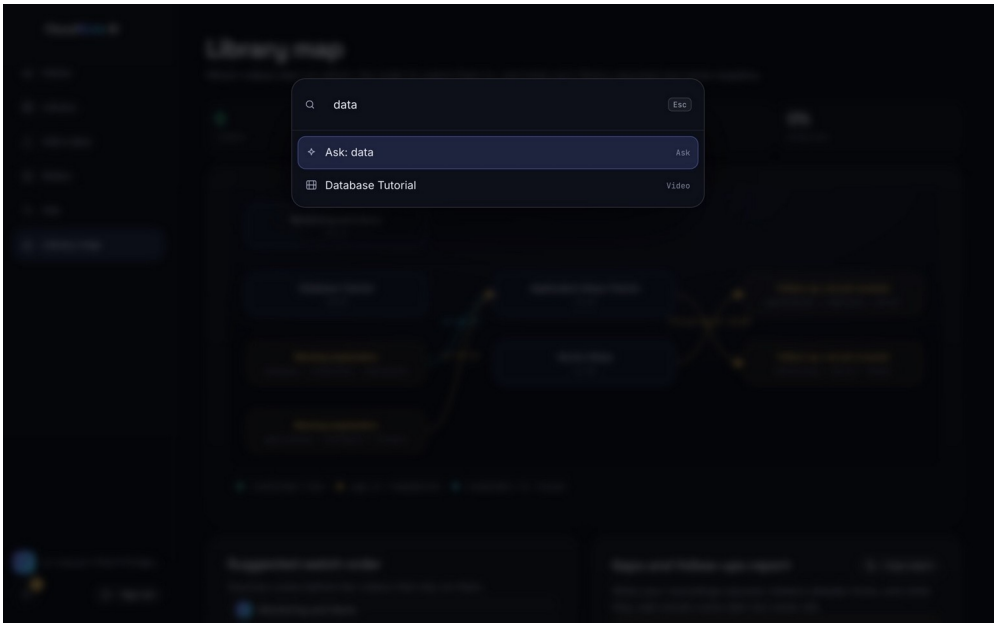


Figure 5.12: Command bar (Ctrl+K)

CHAPTER 6
6. TESTING

6.1 Testing Strategy

Testing was planned against the targets set at the first review: functional correctness, user isolation, security behaviour, grounded AI output, and responsiveness. Table 6.1 summarises the levels of testing.

Table 6.1: Levels of testing

Level	What was tested	Environment
Unit	Parsing, TF-IDF search, reference detection, notes, citation parsing, map	Node test runner
Integration	Every API endpoint with in-memory providers, including isolation and failure paths	Node test runner
System	End-to-end workflows through the browser	Live site, Chrome via Playwright
Security	Authentication, isolation, storage access, headers, input validation, injection	Live deployment
Performance	End-to-end latency, server-side duration, transcription time	Live deployment from India
Usability	Main workflows on desktop and phone widths	Chrome via Playwright; team walkthrough

6.2 Test Environment

Table 6.2: Test environment

Item	Details
Live system	https://d349qer3boe6fg.cloudfront.net - AWS us-east-1; Lambda nodejs22.x (ARM64); DynamoDB on-demand
Language models	Bedrock quota 0, so Groq openai/gpt-oss-120b answered; Gemini as backup
Data	Four illustrative sample transcripts; 11-second speech clip generated with macOS text-to-speech; temporary test accounts deleted after each run
Client	Apple silicon laptop, macOS, located in India; residential broadband (not controlled)
Automated tests	Node.js test runner (3 suites, 59 tests); Python Playwright
Live checks	Python script, 43 checks, run 2026-10-31 04:13:39 UTC

6.3 Functional and Integration Test Cases

The automated suites run on every change. All 59 tests pass. Tables 6.3 to 6.5 list them with their identifiers.

Table 6.3: Engine test cases (tests/engine.test.mjs)

ID	Test case	Result
ENG-01	Detects exactly the two reference chunks in v2	Pass
ENG-02	No false positives in v1 / v3	Pass
ENG-03	Reference 2 (server config) resolves to Server Setup 03:20	Pass
ENG-04	Reference 1 (database settings) has NO suitable source on day 1	Pass
ENG-05	Pending reference resolves to Database Tutorial 04:20 after v3 upload	Pass
ENG-06	Dedupe: same candidate is not alerted twice	Pass
ENG-07	Resolved / rejected refs are skipped	Pass
ENG-08	V3 upload does not wrongly resolve the server-config reference	Pass
ENG-09	Notes vs video: 3 covered, 1 mismatch (8080 vs 8000), 1 missed (restart)	Pass
ENG-10	Notes: adding the restart line flips it to covered	Pass
ENG-11	Notes: fixing the port flips mismatch to covered	Pass
ENG-12	Notes: empty notes => everything missed, no crash	Pass
ENG-13	Free-text sample references behave sensibly	Pass

Table 6.4: Backend core test cases (backend/tests/core.test.mjs)

ID	Test case	Result
CORE-01	Transcribe json -> chunks with times and punctuation	Pass
CORE-02	Transcribe json: long silence splits a chunk	Pass
CORE-03	Srt cues merge into sentence chunks	Pass
CORE-04	Vtt with tags and header	Pass
CORE-05	Plain text with [mm:ss] timestamps	Pass
CORE-06	Plain text without timing -> null starts	Pass
CORE-07	Empty input -> no chunks, no crash	Pass
CORE-08	Extractive notes: summary, points with valid chunk ids, order by time	Pass
CORE-09	Extractive notes: decisions and actions found by pattern	Pass
CORE-10	Extractive notes: empty and single chunk	Pass
CORE-11	Llm reply validation drops invented sources and clamps	Pass
CORE-12	Llm reply with no usable content throws	Pass
CORE-13	GenerateNotes falls back to extractive when the model fails	Pass
CORE-14	Batching splits long transcripts	Pass
CORE-15	Ref items: two backward references and one promise in v2; one promise in v1; none in v3	Pass
CORE-16	Promises: 'in the next recording' in Server Setup is kept by Application Setup; the monitoring promise is not kept by anything	Pass
CORE-17	Promises can only be kept by a later video	Pass
CORE-18	Candidates: 09:30 resolves to Server Setup 03:20; 06:10 has none on day 1	Pass
CORE-19	Pending check: day 2 finds v3-03 once, never twice, skips resolved	Pass
CORE-20	Ask search: finds the moment, rejects off-topic	Pass
CORE-21	Semantic mode: embeddings drive ranking when both sides have them	Pass
CORE-22	Library map: edges, gaps, order, stats	Pass

Table 6.5: API test cases (backend/tests/api.test.mjs)

ID	Test case	Result
API-01	Requires a signed-in user	Pass
API-02	Me reports capabilities and limits	Pass
API-03	Sample library: two videos processed to ready with notes, chunks and 2 refs	Pass
API-04	References: 09:30 has a candidate in Server Setup; 06:10 has none	Pass
API-05	Accept a candidate, remember the other, map reflects both	Pass
API-06	Adding the database sample notifies once; accept resolves; no repeat	Pass
API-07	Promise kept: accepting the later video links it and the map orders the promiser first	Pass
API-08	Promise notification: a pending promise is answered by a later upload, once	Pass
API-09	Ask: finds moments with timestamps; rejects empty; off-topic returns nothing	Pass
API-10	Notes vs video: saves my notes; compare returns verdicts	Pass
API-11	Isolation: user B sees nothing of A and cannot touch it	Pass
API-12	Transcript import: srt becomes a ready video; bad input is rejected	Pass
API-13	Media flow: create -> presigned put -> start -> transcribe event -> ready	Pass
API-14	Transcribe failure marks the video failed; retry uses a new job name; stale and duplicate events are ignored	Pass
API-15	A stuck transcription can be restarted after 30 minutes	Pass
API-16	Deleting a video while it is processing leaves no orphans and no ghost record	Pass
API-17	Unpunctuated text is split into small chunks; re-import replaces old chunks; bad path is a 400	Pass
API-18	Deleting a source video reopens references that pointed into it	Pass
API-19	Library cap is enforced	Pass
API-20	Semantic path: with embeddings, chunks and refs carry vectors and search still works	Pass
API-21	Llm path: model notes are validated and stored; model failure falls back	Pass
API-22	Notes overview lists ready videos with timestamps and own notes, isolated per user	Pass
API-23	Ask notes: keyword mode, grounded AI answers, uncited or failing models fall back	Pass
API-24	Single note page data and per-video ask scope	Pass

6.4 Live System Test Cases

The following checks were executed by a script against the live deployment on 2026-10-31 04:13:39 UTC, using two temporary accounts that were deleted afterwards. All 43 checks passed.

Table 6.6: Authentication test cases (live)

ID	Test case	Expected	Actual	Result
AUTH-01	Unauthenticated GET /api/me is rejected	401	401	Pass
AUTH-02	Unauthenticated GET /api/videos is rejected	401	401	Pass
AUTH-03	Unauthenticated GET /api/notes is rejected	401	401	Pass
AUTH-04	Unauthenticated GET /api/search?q=x is rejected	401	401	Pass

ID	Test case	Expected	Actual	Result
AUTH-05	Unauthenticated GET /api/map is rejected	401	401	Pass
AUTH-06	Unauthenticated GET /api/notifications is rejected	401	401	Pass
AUTH-07	Unauthenticated POST /api/notes/ask is rejected	401	401	Pass
AUTH-08	Forged/tampered JWT is rejected	401	401	Pass
AUTH-09	Sign-up with valid email and password is confirmed immediately	account usable	ok	Pass
AUTH-10	Sign-up with a weak password (5 chars) is rejected by Cognito	InvalidPasswordException	InvalidPasswordException	Pass
AUTH-11	Sign-in with a wrong password is rejected	NotAuthorizedException	NotAuthorizedException	Pass
AUTH-12	Unknown e-mail gives the same error as a wrong password (no enumeration)	NotAuthorizedException	NotAuthorizedException	Pass
AUTH-13	Valid token returns the user's capabilities	200	200 transcription=True	Pass

Table 6.7: Isolation, input validation and injection test cases (live)

ID	Test case	Expected	Actual	Result
ISOL-14	User B cannot read user A's video	404	404	Pass
ISOL-15	User B cannot delete user A's video	404	404	Pass
ISOL-16	User B's library does not list user A's videos	0 videos	0	Pass
ISOL-17	User B's ask-your-notes cannot see user A's notes	no answer, 0 results	answer=False, results=0	Pass
INPU-18	Unsupported file type is rejected	400	400 Unsupported file type. Use: mp4, m4a, mp3, wav, fl	Pass
INPU-19	File larger than 500 MB is rejected	400	400 That file is larger than 500 MB.	Pass
INPU-20	Video without a title is rejected	400	400 Give the video a title.	Pass
INPU-21	Path traversal in the URL does not reach data	4xx	403	Pass
INPU-22	Empty search is rejected	400	400	Pass
INPU-25	Malformed reference id is rejected	4xx	404	Pass
INJE-23	SQL-style injection string in search is treated as text	200, no error	200, 0 results	Pass
INJE-24	Prompt-injection question gets no invented answer	no uncited answer	answer=None	Pass

Table 6.8: Storage, transcription and feature test cases (live)

ID	Test case	Expected	Actual	Result
STOR-26	Upload URL is a short-lived presigned S3 URL	X-Amz-Expires present	X-Amz-Expires=3600	Pass
STOR-27	Unsigned PUT directly to the media bucket is refused	403	403	Pass
STOR-28	Starting transcription before uploading is refused	400	400 The upload did not finish. Try uploading agai	Pass
STOR-29	Presigned PUT upload succeeds	200	200	Pass
STOR-30	A second start (double click) cannot start a second job	400	400 Transcription already started.	Pass
STOR-33	Playback link is a signed, expiring GET URL	X-Amz-Signature present	signed	Pass

ID	Test case	Expected	Actual	Result
TRAN-31	Real speech clip is transcribed by Amazon Transcribe and gets AI notes	ready, model notes	ready, notes=model, 16.4s	Pass
TRAN-32	Follow-up 'in the next video' is detected in the transcribed clip	1 forward reference	1 forward	Pass
FEAT-34	Ask-your-notes answers a covered question with citations	answer with [n]	source=ai, cited=True	Pass
FEAT-35	Ask-your-notes declines an off-topic question	no answer	answer=False	Pass
FEAT-36	Know more returns a labelled general explanation	200, ai-general	200, ai-general, 64 words	Pass
FEAT-37	Library map returns nodes, edges and gaps	200	200, 3 nodes, 5 gaps	Pass
FEAT-38	Search returns timestamped moments	results with start	200, 4 results	Pass

Table 6.9: Transport and header test cases (live)

ID	Test case	Expected	Actual	Result
HEAD-39	Site sends HSTS (1 year, includeSubDomains)	max-age=31536000	max-age=31536000; includeSubDomains	Pass
HEAD-40	Site sends X-Frame-Options DENY and nosniff	DENY, nosniff	DENY, nosniff	Pass
HEAD-41	Site sends a Content-Security-Policy	present	default-src 'self'; script-src 'self'; style-src 'self' 'uns	Pass
HEAD-42	Plain HTTP is redirected to HTTPS	301 to https	301 https://d349qer3boe6fg.cloudfront.net/	Pass
HEAD-43	Website bucket cannot be read directly (only via CloudFront)	403	403	Pass

6.5 Security Testing

- **Authentication.** Seven protected endpoints probed without a token returned 401, and a forged token was rejected. A weak password was refused at sign-up, and a wrong password and an unknown e-mail produced the same error, so sign-in does not reveal which accounts exist.
- **Isolation.** A second user could not read, delete, list or ask about the first user's recordings; reads and deletes returned 404 rather than 403, which also hides whether the recording exists.
- **Storage.** Uploads use presigned URLs valid for one hour; an unsigned PUT to the media bucket and a direct read of the website bucket were refused with 403; playback links are signed. Starting a transcription before uploading and starting it twice were both refused.

- **Injection.** An SQL-style string in search was treated as text, and a prompt-injection question produced no answer. DynamoDB is not queried with string-built queries, and model output is never executed.
- **Headers.** HSTS for one year with subdomains, X-Frame-Options DENY, nosniff and a Content-Security-Policy were present, and plain HTTP was redirected to HTTPS with status 301.

6.6 Performance Testing

6.6.1 End-to-end API latency

End-to-end latency was measured from a client in India to the deployment in us-east-1 over one reused HTTPS connection. The figures therefore include the intercontinental round trip, which alone is about 0.3 s, and the network was unstable during the run (several connections were reset).

Table 6.10: End-to-end API response times from India (ms, reused connection)

Endpoint	n	Min	Median	p95	Max
GET /api/me	40	433	744.6	3,448.6	4,076.6
GET /api/videos	40	477.8	755	2,106.3	3,745.4
GET /api/videos/:id	40	355.9	800.6	4,413.8	7,393.1
GET /api/notes	40	445.7	1,591.1	5,916.7	6,116.6
GET /api/search?q=	40	338	541.4	1,915.1	8,970.4
GET /api/map	40	370.5	553.5	813.8	839.3
GET /api/notifications	40	374.7	570.5	1,377.6	1,606.6
POST /api/notes/ask (AI)	10	1,329.6	1,945.8	10,333.4	10,333.4

6.6.2 Server-side processing time

CloudWatch REPORT records for the measurement hour separate application time from network time. The API function handled 626 invocations with a median duration of 14.9 ms (mean 85 ms, p95 232.5 ms) and at most 135 MB of the 1024 MB configured. There were 2 cold starts with a median initialisation time of 361 ms. Most of the end-to-end latency in Table 6.10 is therefore network transit rather than application work.

Table 6.11: Server-side Lambda duration (CloudWatch, 31 October 2026)

Function	n	Median	p95	Cold starts	Init median
API	626	14.9 ms	232.5 ms	2	361 ms
Worker (notes and indexing)	6	3,283.3 ms	4,781.1 ms	2	451.6 ms

6.6.3 Pipeline timing

Table 6.12: Observed pipeline timing on the live deployment

Operation	Runs	Median	Range
Transcript import to AI notes (29 Oct)	3	4.6 s	4.4 to 4.9 s
Upload, Transcribe and AI notes, 11-s clip (29 Oct)	3	17.3 s	8.4 to 21.2 s
Upload, Transcribe and AI notes, 11-s clip (31 Oct)	1	16.4 s	-
Ask your notes (29 Oct)	10	2.5 s	2.0 to 21.9 s
Know more explanation (29 Oct)	5	4.1 s	2.3 to 15.0 s

6.7 Responsive and Browser Testing

All pages were exercised in Chrome at a desktop viewport of 1440 x 900 and a phone viewport of 390 x 844 using Playwright, with no console errors. At phone width the side menu becomes a bottom tab bar and the bell and sign-out move to the top (Figure 6.1). Formal testing on Safari and Firefox was not carried out and is noted as a limitation.

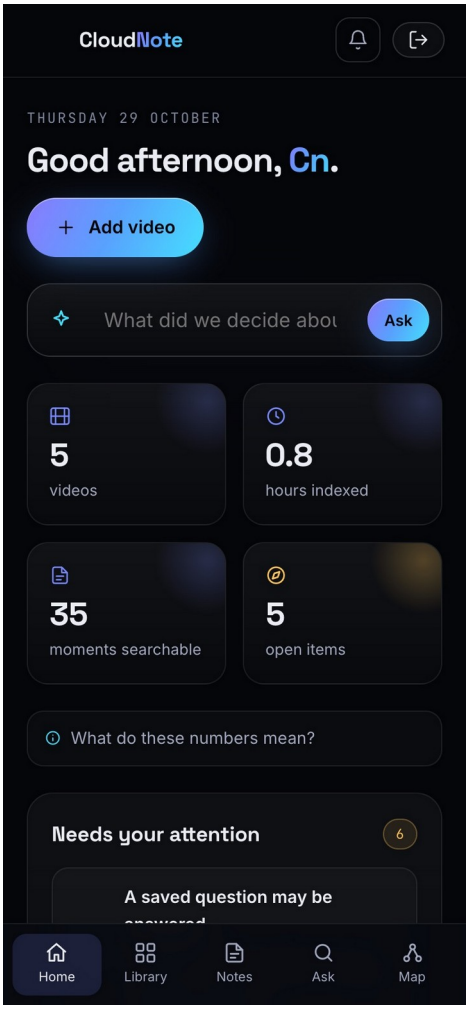


Figure 6.1: Home at 390 px phone width

6.8 Usability Evaluation

The main workflows (sign up, load samples, upload a clip, read notes, ask a question, link a reference, remember a reference and receive the notification, compare notes) were walked through by the team on desktop and phone layouts, and issues found during development were fixed. No formal usability study, such as a System Usability Scale survey with students, was conducted; this is future work.

6.9 Summary of Test Results

Table 6.13: Summary of test results by category

Category	Test cases	Passed	Failed
Engine (unit)	13	13	0
Backend core (unit)	22	22	0
API (integration)	24	24	0
Live: Authentication	13	13	0
Live: Isolation	4	4	0
Live: Input validation	6	6	0

Category	Test cases	Passed	Failed
Live: Injection	2	2	0
Live: Storage	6	6	0
Live: Transcription	2	2	0
Live: Features	5	5	0
Live: Headers	5	5	0
Total	102	102	0

During the first real deployment six defects appeared that local tests could not reveal; they are listed in Section 7.2 and were all fixed before the final test runs.

CHAPTER 7
7. RESULTS AND DISCUSSION

7.1 Overview of Results

The outcome of the project is a complete, deployed system at https://d349qer3boe6fg.cloudfront.net with public source code. All 59 automated tests and all 43 live checks passed. The live system transcribed a real speech clip with Amazon Transcribe, wrote AI notes, detected the follow-up phrase it contained, answered covered questions with citations, declined off-topic questions, and isolated users from each other.

7.2 Discussion of Measured Results

Latency. Server-side, the API function's median time was 14.9 ms. Seen from India, median end-to-end times over a reused connection were between 541.4 and 1,591.1 ms for reads, dominated by the round trip to us-east-1 and an unstable network during measurement. Deploying in ap-south-1 (Mumbai) would remove most of this delay for Indian users; us-east-1 was chosen because Transcribe and Bedrock model availability were broadest there.

AI answers. Answers took about 2 to 3 s typically. Long tails (up to about 22 s) occurred when the free Groq tier rate-limited a request and the chain moved to the next provider; the fallback worked as designed, but free-tier access is not dependable under bursts.

Transcription. An 11-second clip took 8.4 to 21.2 s from upload to notes, mostly job scheduling inside Transcribe. The transcript was accurate except that "DynamoDB" was written as "Dynamo DB", and the follow-up phrase was detected in every run.

Table 7.1: Defects found during the first real deployment

Symptom	Cause and fix
Sign-up failed	Packaging ignored a global code path, so functions shipped without code; set per function
Database writes failed	Missing dynamodb:UpdateItem permission; added to both function roles
Transcription refused	The AWS Free plan excludes Transcribe; account upgraded to the paid plan with credits retained
Notes always extractive	Bedrock quota of zero; provider fallback chain added
Gemini fallback failed	Older models retired for new keys, current model overloaded; time limits and retries added

Symptom	Cause and fix
AI answers discarded	Unrecognised citation formats; parser extended and logging added

7.3 Operating Cost

AWS Cost Explorer reported a total of 0.00 USD for the project account for October 2026, covering development, repeated deployments, test accounts and Transcribe jobs; usage stayed within free allowances and promotional credits. Table 7.2 estimates transcription cost, the dominant item, for larger use, assuming the standard batch price of about 0.024 USD per audio minute in us-east-1 at the time of writing. Prices change and should be checked before budgeting.

Table 7.2: Estimated transcription cost by usage scenario

Scenario	Audio minutes	Approx. cost (USD)
Capstone demonstration: 4 recordings of 5 minutes	20	0.48
One student, one course: 30 lectures of 50 minutes	1,500	36
60 students each uploading the same 30 lectures	90,000	2,160
Same class with lectures uploaded once and shared	1,500	36

Lambda, DynamoDB, API Gateway, S3, CloudFront and Cognito cost near zero at this scale because they are billed per request and stored gigabyte. The scenarios show that a shared course library, or importing existing captions as transcripts, is the key to class-scale affordability.

7.4 Achievement of Objectives

Table 7.3: Objectives, implementation and evidence

Objective	Implementation	Evidence
1. Timestamped transcription	Transcribe + EventBridge; four transcript formats	TRAN-31; core parsing tests
2. Verifiable notes	Model notes tied to chunk numbers; extractive fallback	CORE notes tests; Figure 5.2
3. Grounded answers	Citations required; uncited answers dropped; Ask tab answers with moments	FEAT-34, FEAT-35, INJE-24
4. Context recovery	Patterns, candidates, remembered references, notifications, map	ENG and API reference tests; FEAT-37
5. Secure serverless deployment	Cognito, JWT authorizer, per-user partitions, private S3, IaC	AUTH, ISOL, STOR, HEAD checks
6. Honest evaluation	Live tests, CloudWatch timings, documented limits	Chapter 6; Section 7.6

7.5 Comparison with Surveyed Approaches

Table 7.4: Qualitative comparison with surveyed approaches

Dimension	Video navigation [1], [2]	Summarisation [5]-[7]	RAG and citations [12], [14]	CloudNote AI
Scope	One video	One document	Large corpus	Learner's library of recordings
Timestamps	Yes	No	No	Every note and answer
Cross-recording links	No	No	No	Detected, suggested, confirmed
Memory for missing explanations	No	No	No	Re-checked after each upload
Verification	Viewer data	Faithful if extractive	Citations, often incomplete	Citations enforced and shown
Deployment	Research prototype	Algorithms	Models and benchmarks	Live on AWS, open source

7.6 Limitations

- **Reference recovery not yet scored.** No labelled set of real lecture series exists yet for this task; the sample recordings were written to contain clear references, so success on them does not show generalisation.
- **English-only patterns.** Transcription supports Hindi, Telugu and other languages, but the reference patterns are English.
- **Semantic search disabled.** Titan embeddings are implemented but cannot run while the Bedrock quota is zero, so paraphrases are missed.
- **Third-party models.** While Groq or Gemini answers, transcript text leaves AWS, and free tiers rate-limit bursts.
- **Region latency.** The us-east-1 deployment adds about 0.3 s per request for users in India.
- **Open sign-up, no password reset or e-mail verification** in the current deployment.
- **Small samples** for performance measurements, from one client location; no formal usability or learning study.
- **Per-container in-memory index** suits libraries of a few hundred recordings per user.

7.7 Answer to the Research Question

The research question asked how a cloud platform can turn recorded lectures into searchable, verifiable notes and recover the context that recordings leave out. The results indicate that this can be achieved by combining (i) managed speech recognition that preserves timestamps, (ii) language-model notes and answers that must cite numbered moments, with an extractive fallback, (iii) transparent reference detection whose suggestions the learner confirms, (iv) remembered references re-checked as the library grows, and (v) a serverless, per-user isolated architecture defined as code. How accurately this works on real, unscripted lecture series is the question for the next stage.

CHAPTER 8
8. CONCLUSION AND FUTURE SCOPE

8.1 Conclusion

This project set out to make a library of recorded lectures searchable, verifiable and aware of the links between its recordings. The answer delivered is CloudNote AI, a working, deployed system. It transcribes recordings into timestamped moments, writes notes whose every item points to its moment, answers questions only from the learner's own material with citations and playable timestamps, detects references and follow-ups between recordings, and keeps watching for explanations that are not yet in the library.

The system runs on a secure, serverless AWS stack with authentication enforced before any code runs, per-user data isolation, private storage, and infrastructure defined as code. The evaluation showed that the functional requirements were met, that isolation and security controls behaved as designed under deliberate misuse, and that the application itself responds in milliseconds, with end-to-end time dominated by the network. The work also made its limitations explicit.

In relation to the literature, the contribution is integration around a gap: established techniques for transcription, retrieval, summarisation and grounded generation are combined into an inspectable system for cross-video context recovery that others can run and audit.

8.2 Future Scope

1. **Measured reference recovery.** Build a labelled dataset from several real lecture series and measure detection recall and candidate precision.
2. **Enable Bedrock and Titan embeddings** once the quota is raised, and tune the semantic thresholds (0.50 for references, 0.33 for questions) on real data.
3. **Indian languages.** Add Hindi and Telugu reference patterns to match the transcription languages already supported.
4. **Course libraries.** Shared libraries per course so a lecture is transcribed once for all students, with instructor gaps reports.
5. **Mumbai region deployment** to cut latency for users in India.
6. **Production safeguards:** restricted sign-up, e-mail verification, password reset, budget alarms and rate limits.

7. **BM25 and managed vector search** for larger libraries.
8. **User study** measuring whether linked recordings reduce the time students need to recover missing context.

REFERENCES

[1] P. J. Guo, J. Kim, and R. Rubin, "How video production affects student engagement: An empirical study of MOOC videos," in *Proc. 1st ACM Conf. Learning @ Scale (L@S '14)*, 2014, pp. 41-50.

[2] J. Kim, P. J. Guo, C. J. Cai, S.-W. Li, K. Z. Gajos, and R. C. Miller, "Data-driven interaction techniques for improving navigation of educational videos," in *Proc. 27th ACM Symp. User Interface Software and Technology (UIST '14)*, 2014, pp. 563-572.

[3] R. E. Mayer and R. Moreno, "Nine ways to reduce cognitive load in multimedia learning," *Educational Psychologist*, vol. 38, no. 1, pp. 43-52, 2003.

[4] A. Radford, J. W. Kim, T. Xu, G. Brockman, C. McLeavey, and I. Sutskever, "Robust speech recognition via large-scale weak supervision," in *Proc. 40th Int. Conf. Machine Learning (ICML)*, PMLR vol. 202, 2023.

[5] G. Erkan and D. R. Radev, "LexRank: Graph-based lexical centrality as salience in text summarization," *Journal of Artificial Intelligence Research*, vol. 22, pp. 457-479, 2004.

[6] R. Mihalcea and P. Tarau, "TextRank: Bringing order into texts," in *Proc. Conf. Empirical Methods in Natural Language Processing (EMNLP)*, 2004, pp. 404-411.

[7] A. Nenkova and K. McKeown, "A survey of text summarization techniques," in *Mining Text Data*, C. C. Aggarwal and C. Zhai, Eds. Springer, 2012, pp. 43-76.

[8] G. Salton and C. Buckley, "Term-weighting approaches in automatic text retrieval," *Information Processing & Management*, vol. 24, no. 5, pp. 513-523, 1988.

[9] S. Robertson and H. Zaragoza, "The probabilistic relevance framework: BM25 and beyond," *Foundations and Trends in Information Retrieval*, vol. 3, no. 4, pp. 333-389, 2009.

[10] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using Siamese BERT-networks," in *Proc. EMNLP-IJCNLP*, 2019, pp. 3982-3992.

[11] V. Karpukhin et al., "Dense passage retrieval for open-domain question answering," in *Proc. EMNLP*, 2020, pp. 6769-6781.

[12] P. Lewis et al., "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Advances in Neural Information Processing Systems*, vol. 33, 2020, pp. 9459-9474.

[13] Z. Ji et al., "Survey of hallucination in natural language generation," *ACM Computing Surveys*, vol. 55, no. 12, art. 248, 2023.

[14] T. Gao, H. Yen, J. Yu, and D. Chen, "Enabling large language models to generate text with citations," in *Proc. EMNLP*, 2023.

[15] E. Jonas et al., "Cloud programming simplified: A Berkeley view on serverless computing," Univ. of California, Berkeley, Tech. Rep. UCB/EECS-2019-3, 2019.

[16] C. D. Manning, P. Raghavan, and H. Schütze, *Introduction to Information Retrieval*. Cambridge University Press, 2008.

[17] OpenAI, "gpt-oss-120b & gpt-oss-20b model card," Aug. 2025.

[18] Amazon Web Services, "Amazon Transcribe developer guide." [Online]. Available: https://docs.aws.amazon.com/transcribe/

[19] Amazon Web Services, "Amazon Bedrock user guide." [Online]. Available: https://docs.aws.amazon.com/bedrock/

[20] Amazon Web Services, "Amazon Cognito developer guide." [Online]. Available: https://docs.aws.amazon.com/cognito/

[21] Amazon Web Services, "Amazon DynamoDB developer guide." [Online]. Available: https://docs.aws.amazon.com/dynamodb/

[22] Amazon Web Services, "AWS Serverless Application Model developer guide." [Online]. Available: https://docs.aws.amazon.com/serverless-application-model/

[23] OWASP Foundation, "OWASP Top 10: 2021." [Online]. Available: https://owasp.org/Top10/

APPENDIX A: API ENDPOINT REFERENCE

All endpoints are served under /api, accept and return JSON, and require a valid Cognito ID token in the Authorization header; API Gateway rejects requests without one. Table A.1 lists the 28 endpoints.

Table A.1: REST API endpoints

Method	Path	Purpose
GET	/me	User id, limits and enabled capabilities
GET	/videos	List the user's recordings
POST	/videos	Create a recording; returns a presigned upload URL for media
GET	/videos/{id}	Recording with notes
PATCH	/videos/{id}	Rename
DELETE	/videos/{id}	Delete recording, media and transcript
GET	/videos/{id}/transcript	Timestamped chunks
POST	/videos/{id}/transcript	Import a transcript
POST	/videos/{id}/start	Start transcription after upload
POST	/videos/{id}/reprocess	Regenerate notes
GET	/videos/{id}/media	Signed playback URL
GET	/videos/{id}/references	References with candidates
GET	/videos/{id}/mynotes	User's own notes
PUT	/videos/{id}/mynotes	Save own notes
POST	/videos/{id}/compare	Compare own notes with key points
GET	/search?q=&video=	Moments and an AI answer from the transcript
POST	/references/{id}/accept	Link a source
POST	/references/{id}/remember	Remember an unresolved reference
POST	/references/{id}/undo	Reset a reference
GET	/notifications	Notifications and unread count
POST	/notifications/{id}/accept	Accept a suggested link
POST	/notifications/{id}/reject	Dismiss a notification
GET	/map	Library map, watch order, gaps
POST	/notes/ask	Ask your notes (grounded, cited)
GET	/notes/{id}	Notes of one recording
GET	/notes	Notes of all ready recordings
POST	/samples	Add sample recordings
POST	/explain	Know more: general explanation

APPENDIX B: DEPLOYMENT AND RUN GUIDE

B.1 Running locally

Requirements: Node.js 20 or later. No AWS account is needed; sign-in is a development login, files are stored on disk and uploads receive a sample transcript.

Listing B.1: Local setup

```
git clone https://github.com/kmithileshhhhh05/cloudnote-ai.git
cd cloudnote-ai
npm install
node build.mjs
DATA_DIR=/tmp/cn-data node backend/local-server.mjs   # http://localhost:8787

node tests/engine.test.mjs && node backend/tests/core.test.mjs && node
backend/tests/api.test.mjs
```

B.2 Deploying to AWS

Requirements: the AWS CLI configured with credentials allowed to create the stack, and Node.js. Model API keys are read from files and passed as hidden parameters.

Listing B.2: Deployment commands

```
ALLOWED_EMAIL_DOMAINS="" BEDROCK_REGION=us-west-2 EMBED_MODEL_ID=none \
  GROQ_API_KEY_FILE=~/.cloudnote/groq.key
GEMINI_API_KEY_FILE=~/.cloudnote/gemini.key \
  bash infra/deploy.sh us-east-1      # first deploy or update

bash infra/destroy.sh us-east-1       # remove everything (asks before deleting
data)
```

B.3 Recovery

The DynamoDB table has point-in-time recovery enabled and is retained if the stack is deleted, as is the media bucket. A table can be restored to any second within the recovery window from the DynamoDB console, after which the stack's table parameter is pointed at the restored table.

APPENDIX C: USER GUIDE

C.1 Getting started

- Open https://d349qer3boe6fg.cloudfront.net, choose Create an account, and sign up with e-mail and a password of 8+ characters including a number.
- Click Try with sample videos to load two sample lectures; add the other two from the Library page.

C.2 Everyday tasks

- **Add a recording:** Add video, choose a file or paste a transcript, pick the language, submit. The page fills in when transcription finishes.
- **Ask about a video:** open it, choose the Ask tab, type a question. You get an answer from the transcript and the moments it came from.
- **Ask your notes:** Notes page, type a question; the answer cites your notes.
- **Follow a reference:** Context tab, accept a suggested source, pick one manually, or remember it. The bell shows a notification when a later upload explains it.
- **See the big picture:** Library map shows dependencies, a watch order and a gaps report.
- **Command bar:** press Ctrl+K (Cmd+K on Mac) anywhere.

APPENDIX D: PROJECT LINKS

Table D.1: Project links

Item	Link
Live system	https://d349qer3boe6fg.cloudfront.net
Source code (GitHub)	https://github.com/kmithileshhhhh05/cloudnote-ai
User manual	docs/manual/CloudNote_AI_User_Manual.pdf in the repository
Cloud region	AWS us-east-1 (N. Virginia)
Team contacts	2320030330@klh.edu.in, 2320030186@klh.edu.in, 2320030348@klh.edu.in